Data, Decisions and Models

Lecture Notes

Master of Global Manufacturing and Supply Chain Management (GMSCM)

Version: 2026 **Document Type:** Course Notes

Data, Decisions and Models

Lecture Notes

These notes were prepared for the Master of Global Manufacturing and Supply Chain Management (GMSCM) Program.

Contents

1 | Course Structure

This chapter covers how the course is organized, and what is expected of students.

1.1 Course Format and Schedule

MGSC 608 Data, Decisions and Models is a foundation course in statistics and optimization for the Master of Global Manufacturing and Supply Chain Management (GMSCM) Program.

1.2 Assessment and Assignments

Assessment includes three assignments. Assignments 1 and 3 are group assignments, while Assignment 2 is individual.

Teams should ideally contain four students, although smaller teams are acceptable. Students may form teams across different sections, since the sections are treated as one combined class. One team member must complete the final submission before the deadline; merely placing a file in the assignment area may not count as a completed submission. Deadlines are strict and indicated on myCourses.

The first group assignment uses the Merton Truck Company case, and the final group assignment uses the Freemark Abbey Winery case. These are the two mandatory readings; the suggested textbooks are optional references.

1.3 Software: Python and Excel

The course is programming-language agnostic at the conceptual level. That being said, python is the default implementation language and is strongly encouraged, since it will matter throughout the program. Excel is also acceptable, especially for students transitioning into Python; Excel users should activate Solver and the Analysis ToolPak. The key message is that software is a means of implementing optimization, not the subject of the course itself.

Students are given a shared online Python workspace containing examples organized by lesson. They should fork the workspace to create an editable personal copy, experiment with the code, and practice. A local Python setup (for example, an editor such as Visual Studio Code) is another option.

1.4 The Final Exam

The final exam is a three-hour, in-class, conceptual exam. Computers are not allowed, and students will not be asked to reproduce Python, Excel, or other programming syntax. Calculators are allowed, as is one double-sided page of notes, and any calculations should be relatively simple. A practice exam from a previous year illustrates the expected style of questions.

Part I

Optimization Foundations (Module 1)

2 | Optimization in Analytics and Machine Learning

2.1 Training as Optimization

Optimization is a core mechanism behind modern analytics, machine learning, and artificial intelligence. Every machine-learning model must be trained: training means adjusting model parameters so that the model's output moves closer to a known target, often called the ground truth or label. Behind the scenes, that adjustment is an optimization problem. The model begins with imperfect predictions, measures the error, and repeatedly changes its parameters to reduce that error.

The training loop can be stated in business terms: a model produces an output, the output is compared with what actually happened, and the difference is converted into a numerical loss. The optimizer then changes the model parameters and tries again, and this sequence may be repeated thousands or millions of times. Training is therefore not a mysterious feature added to a model; it is a repeated decision process governed by an objective.

2.2 Classification and Regression

Two broad machine-learning tasks illustrate this idea. *Classification* predicts a discrete category, such as whether an image is a cat or a dog, or which token should come next in a language model, and commonly uses a cross-entropy objective. *Regression* predicts a continuous quantity, such as a future price, and commonly uses squared error.

For classification, the output is one of a finite set of possibilities. A language model's next-token choice feels open-ended, but it is still a classification over a discrete vocabulary; cross-entropy translates the quality of the predicted probability distribution into a loss, penalizing a correct label that receives low predicted probability.

For regression, the target lies on a continuous scale (a future stock price, for example). Squared error makes both positive and negative residuals cost something and gives greater weight to large misses. In both cases, training searches for parameter values that improve an objective.

2.3 Optimization Beyond Machine Learning

Optimization is much broader than machine learning. In supply chains, a company such as Amazon must choose which fulfillment center should serve an order, whether to reserve inventory for future demand, and which transportation mode to use. A nearby

warehouse may be cheaper for the present order but strategically valuable for another customer; trucks and aircraft have different costs and capacity limits. These competing considerations turn fulfillment into an optimization problem.

Retailers optimize prices and promotions: when to offer a discount, how deep it should be, and whether to customize an offer using a customer's history. Manufacturers decide what to produce, in what quantities, and how much inventory to hold. The business domain changes, but the mathematical structure is similar.

2.4 The Three Components of an Optimization Model

Every optimization model has three essential components.

Decision variables are the quantities the model is allowed to choose. In a pricing model, the decision variable may be price; in an AI model, the variables are the trainable parameters, potentially numbering in the billions. More variables do not automatically create a better model — a model must balance expressive power, training cost, deployment feasibility, energy use, and reliability.

The objective function evaluates how good a candidate solution is (minimizing prediction error, minimizing fulfillment cost, maximizing profit). A useful objective should be simple enough to calculate and optimize, yet rich enough to capture the central trade-off in the real problem.

Constraints translate business rules and physical limits into mathematical conditions: a transportation plan cannot exceed vehicle capacity, a price may need to stay within an approved range, certain promotions may not be offered together, model parameters may need to be nonnegative or bounded. Constraints define the feasible space — the set of solutions that are allowed and implementable.

2.5 Formulation Requires Judgment

An optimization system can solve only the model it is given. If the decision variables omit a crucial choice, if the objective rewards the wrong outcome, or if a business rule is missing, a mathematically optimal answer may be operationally poor. This is why students are asked to compare models generated by AI tools and evaluate their realism rather than accepting them uncritically.

The fulfillment and pricing examples both illustrate why an apparently obvious choice may not be optimal: serving a customer from the nearest warehouse reduces distance, but that warehouse's inventory may be more valuable for another order, and a lower price may increase demand while reducing margin. These examples prepare students to look for trade-offs rather than search for a rule that says one action is always best.

3 | Variables, Objectives, Constraints, and Convexity

3.1 Convexity of Feasible Regions

A feasible region is *convex* when the straight line joining any two feasible points stays entirely inside the region. Convexity matters because optimization algorithms move through a sequence of candidate solutions, and in a convex region movement between feasible solutions behaves predictably. Non-convex regions can contain gaps, isolated areas, or multiple hills and valleys, making them harder and more expensive to search. Linear constraints are especially useful because they generate convex feasible regions and lead to scalable solution methods.

The key modeling discipline is: identify what can be chosen, define exactly what success means, express every important limitation, and then check whether the resulting model is computationally manageable and usable in the real world. Optimization is not simply pressing a solve button — it is the careful translation of a decision problem into variables, an objective, and constraints.

3.2 Cross-Entropy Loss for Classification

For supervised classification with observations indexed by i and classes indexed by c, a common cross-entropy loss is

$$L(\theta) = -\sum_{i}\sum_{c} y_{i,c} \log p_{i,c}(\theta).$$

The parameter vector θ contains the model weights. The indicator $y_{i,c}$ equals one for the correct class and zero otherwise, and the predicted probability $p_{i,c}(\theta)$ depends on θ. Minimizing L assigns high probability to correct labels. A language model uses the same basic structure at the token level: the correct next token supplies the label, while the model distributes probability over its vocabulary.

3.3 Mean Squared Error for Regression

For regression, a standard mean-squared-error objective is

$$\mathrm{MSE}(\theta) = \frac{1}{n}\sum_{i=1}^{n}\Big[y_i - \hat{y}_i(\theta)\Big]^2.$$

The residual is actual minus predicted. Squaring makes positive and negative residuals contribute positively and penalizes large deviations more heavily. Training seeks $\theta^\star \in \arg\min L(\theta)$ (or $\arg\min \mathrm{MSE}(\theta)$). Regularization can be added — for example $\lambda\|\theta\|^2$ — when the analyst wants to discourage overly large parameter values.

3.4 The Gradient Descent Update Rule

A differentiable unconstrained minimization method uses gradient descent:

$$\theta_{k+1} = \theta_k - \alpha_k \nabla L(\theta_k).$$

The gradient is the vector of partial derivatives. The minus sign is used because the goal is minimization; for maximization, the update uses a plus sign. The learning rate α_k is the step size and may be fixed, scheduled, or adaptive.

3.5 Formal Definition of a Convex Set and a Convex Function

A set X is convex if, for every $x, z \in X$ and every $\lambda \in [0, 1]$, the weighted average remains feasible:

$$\lambda x + (1 - \lambda)z \in X \quad \text{for all } \lambda \in [0, 1].$$

A function f is convex when the graph lies below every chord joining two points on the graph:

$$f\Big(\lambda x + (1 - \lambda)z\Big) \leq \lambda f(x) + (1 - \lambda) f(z).$$

For twice-differentiable f, a sufficient and (on an open convex domain) equivalent test is that the Hessian matrix is positive semidefinite everywhere: $v^\top H(x)\, v \geq 0$ for every vector v, where $H(x)$ contains all second partial derivatives. For maximizing a concave function, the Hessian is negative semidefinite.

3.6 Linear Inequalities and the Convexity of Linear Programs

A linear inequality $a^\top x \leq b$ defines a half-space, which is convex. Intersections of convex sets remain convex, so the feasible region of a linear program is convex. If the objective is also linear, any local optimum is globally optimal, although multiple globally optimal solutions can exist along an edge or face.

3.7 First-Order Optimality and the KKT Conditions

First-order optimality for a differentiable unconstrained convex minimization problem is $\nabla f(x^\star) = 0$. With constraints, the correct conditions involve feasible directions or the Karush–Kuhn–Tucker (KKT) conditions. In simplified form, KKT combines primal feasibility, dual feasibility, stationarity, and complementary slackness — this distinction explains why an optimum on a boundary can have a nonzero ordinary gradient.

3.8 Decision Variables versus Inputs

Decision variables must be controllable choices, not merely facts in the environment. A retailer can choose a price or an allocation quantity, but cannot directly choose customer demand — demand is predicted as a consequence of price. Similarly, an AI model's parameters are decision variables during training, while the labeled examples are inputs. Clearly separating choices from inputs is one of the first tests of a sound formulation.

3.9 Coefficients, Right-Hand Sides, and the Feasible Set as an Intersection

Objective coefficients express how a unit of a decision contributes to the performance measure; constraint coefficients express how a unit consumes a resource or affects a business rule; right-hand-side values usually represent limits, requirements, or available capacities.

Constraints can be simple bounds (for example, requiring a price to fall between ten and forty) or complex nonlinear relationships. A collection of constraints acts jointly: the feasible set is the intersection of the solutions allowed by every rule, and satisfying one rule is not enough — an implementable solution must satisfy them all at once.

3.10 Convexity in Practice

The geometric definition of convexity does not depend on the number of variables: choose any two feasible solutions, form a weighted average, and if every such average is feasible, the set is convex. In one dimension, a single uninterrupted interval is convex, while two separated intervals are not; in two dimensions, a filled circle or ellipse is convex, while a crescent or a region with a hole is not.

Calculus can test convexity through second derivatives and semidefinite curvature conditions, but explicitly forming a large second-derivative matrix may be computationally unrealistic. Practitioners often build models from known convex components — linear inequalities are a fundamental example — and combine functions using rules that preserve convexity, rather than attempting a brute-force proof for a model with billions of variables.

Convex objectives and feasible sets are attractive because local improvement methods have stronger guarantees. Modern deep neural networks are generally non-convex, which is one reason their training behavior is more complicated; this intuition should be kept distinct from a universal mathematical claim.

4 | The Pricing Model and Gradient Calculations

4.1 A One-Variable Pricing Problem

The lecture develops the basic ideas through a one-variable pricing problem. A sportswear retailer wants to choose the price of a sweatshirt. Because there is only one decision variable, price, this is called *scalar* optimization; a scalar is a single number, while a vector contains several decision variables.

The retailer's estimated demand is $20 - 0.5p$, so demand falls as price rises; the coefficient 0.5 represents price sensitivity. Each unit sold earns a margin equal to the selling price minus a unit cost of \$2, so expected profit is demand multiplied by unit margin: $(20 - 0.5p)(p - 2)$.

The price cannot be below \$10, and the demand formula should not predict negative demand — demand reaches zero at $p = 40$, so price should not exceed \$40. The feasible set is the interval $[10, 40]$, which is convex, since every point between any two feasible prices is also feasible.

4.2 Gradient Search: The Core Idea

To solve the problem, the lecture introduces gradient search, the basic family of methods behind much of modern model training. The algorithm begins with an initial solution (a price of 30), then calculates an update with two parts: a direction supplied by the gradient and a distance controlled by the step size. The new solution equals the current solution plus that update.

In one dimension, the gradient is the derivative of the objective function: it measures how quickly the objective changes when the decision variable is changed by a very small amount. A positive derivative means moving toward a higher value of the decision variable locally increases the objective; a negative derivative means the opposite. At $p = 30$ the derivative is negative, so profit can be improved by reducing the price.

With several decision variables, the gradient becomes a vector of partial derivatives, and each component measures the local rate of change in one direction. This is why gradients scale to models with enormous numbers of parameters: each iteration uses local rate-of-change information rather than evaluating every possible solution.

4.3 Why "Local" Matters

A gradient describes what happens in a small neighborhood around the current point — it does not promise that moving a long distance in the same direction remains beneficial. The step size limits how far the algorithm travels before recalculating the gradient. A very small step is safe but may require millions of iterations; a large step can overshoot the best region or cause repeated zigzagging. Practical methods seek a useful balance.

4.4 Numerical Illustration

In the spreadsheet illustration, the initial price is 30 and a fixed step size of 0.1 is used. The negative gradient produces a negative update, so the price moves downward; the calculation is repeated (recompute the derivative, multiply by the step size, update the price, and continue), and the sequence approaches a price of approximately \$21. At the optimum, the gradient is zero, or close enough to zero that further improvement is negligible.

Expanding the pricing objective, profit $= (20 - 0.5p)(p - 2)$, is a quadratic expression. At low prices, margin is small even if demand is high; at very high prices, margin per unit is high but demand approaches zero. The best price balances these effects, which is why neither the lowest nor the highest allowed price is automatically optimal.

4.5 The Product Rule and the Sign of the Update

The derivative is calculated using the product rule: differentiate the demand term, keep the margin term, then add the demand term multiplied by the derivative of margin. At $p = 30$, the result is negative, indicating that reducing price should improve profit near 30. After the price changes, the derivative must be recalculated, because the objective is curved and its slope changes with location.

The update rule is recursive: if the current price is p_0, the next price is

$$p_1 = p_0 + \alpha \cdot (\text{gradient at } p_0).$$

With a fixed step size $\alpha = 0.1$ and a gradient of -9 at $p_0 = 30$, the first update is -0.9, moving the price from 30 to 29.1. Repeating the calculation generates a sequence that approaches 21.

4.6 Direction of Ascent versus Descent

For a maximization problem, moving in the gradient direction produces a local increase. For a minimization problem, the sign is reversed and the algorithm moves against the gradient — this is usually called *gradient descent*. The broader phrase *gradient search* is used because the same local-derivative idea supports either maximization or minimization; students should pay attention to the stated objective before interpreting the update direction.

The formal stopping condition at an unconstrained interior optimum is a zero gradient. Computers normally use a tolerance (for example, stopping when the gradient magnitude

is below 0.01), since exact zero may be unnecessary or impossible with finite numerical precision. A constrained optimum may occur at a boundary, where the unconstrained gradient need not be zero; constrained solvers account for the allowed directions.

The pricing problem could be solved directly by setting its derivative to zero, and a student raises exactly this point. The instructor agrees, but explains that direct algebra is easy for one or a few variables and does not scale to a model with millions or billions of decisions — the toy problem makes the logic of scalable optimization visible. In real software, users normally specify a convergence tolerance instead of waiting for the derivative to become exactly zero; Excel Solver's generalized reduced gradient method performs this iterative work automatically and can respect lower and upper bounds.

Modern training often uses adaptive step sizes. The Adam family of methods changes the effective step size using information about recent gradients and momentum; the details, along with backpropagation (an efficient way to calculate gradients through neural networks), are deferred to later courses. PyTorch is identified as a commonly used Python package that makes these methods convenient.

5 | Solver Methods, Curvature, and Local Optima

5.1 Solving the Pricing Problem Exactly

The pricing example can be solved completely. Let p be price in dollars, demand in thousands of units be $D(p)$, and unit cost be \$2:

$$D(p) = 20 - 0.5p, \qquad 10 \le p \le 40.$$

$$f(p) = D(p)(p - 2) = (20 - 0.5p)(p - 2).$$

Expanding gives a concave quadratic:

$$f(p) = -0.5p^2 + 21p - 40.$$

Its first and second derivatives are

$$f'(p) = 21 - p, \qquad f''(p) = -1.$$

Because the second derivative is negative, f is strictly concave. The stationary point satisfies $21 - p = 0$, so $p^\star = 21$. This point lies inside the feasible interval, making it the unique global maximum. Demand is 9.5 thousand units, margin is \$19 per unit, and maximum profit is \$180.5 thousand under the model. At $p = 30$, profit is \$140 thousand and the derivative is -9, confirming that price should move downward.

5.2 Convergence of Gradient Ascent

With gradient ascent and fixed step α, the update becomes

$$p_{k+1} = p_k + \alpha(21 - p_k) = (1 - \alpha)p_k + 21\alpha.$$

Subtracting the optimum from both sides gives $p_{k+1} - 21 = (1 - \alpha)(p_k - 21)$. Convergence occurs when $|1 - \alpha| < 1$, equivalently $0 < \alpha < 2$. With $\alpha = 0.1$, the error shrinks by 10% each iteration; from $p_0 = 30$, the first value is 29.1, matching the spreadsheet demonstration.

5.3 Projected Gradient Updates

If an update crosses a bound, a projected method uses

$$p_{k+1} = \mathrm{Proj}_{[10,40]}\big(\text{unconstrained update}\big).$$

Excel's generalized reduced gradient method handles constraints more generally by working with feasible or reduced directions.

5.4 Gradients and Directional Derivatives in Higher Dimensions

For a multivariable objective $f(x)$, the gradient is

$$\nabla f(x) = \left[\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_n}\right]^\top.$$

The directional derivative along a unit vector d is $\nabla f(x)^\top d$. By the Cauchy–Schwarz inequality, this quantity is largest when d points in the gradient direction — the mathematical reason the gradient gives the direction of steepest local ascent under ordinary Euclidean distance.

5.5 The Hessian and Newton's Method

The Hessian describes local curvature:

$$H(x) \text{ has entry } H_{i,j} = \frac{\partial^2 f}{\partial x_i \, \partial x_j}.$$

A Newton step for minimization solves

$$H(x_k)\, d_k = -\nabla f(x_k), \qquad x_{k+1} = x_k + d_k,$$

possibly with a damped step. This rescales the gradient using curvature, which can reduce zigzagging, but storing and solving with an $n \times n$ Hessian is costly when n is large.

5.6 Two-Phase Feasibility Methods

A generic two-phase feasibility method defines a nonnegative violation function $V(x)$. For inequalities $g_i(x) \le 0$, one option is

$$V(x) = \sum_i \max(0, g_i(x))^2 + \sum_j h_j(x)^2.$$

Phase one minimizes V until it is approximately zero; phase two then optimizes the true objective while maintaining feasibility. The $\max(0, g_i(x))$ term is needed so that only a positive value — representing violation of a less-than-or-equal constraint — is penalized.

5.7 Multi-Start, Smoothing, and Local Optima

At a local minimum, the gradient may be zero even when another, lower basin exists. Multi-start samples several initial vectors and compares the resulting local solutions, keeping the best. Smoothing or continuation methods replace the original objective with an easier family f_τ and gradually change the temperature parameter τ. Both strategies spend additional computation to reduce dependence on one local starting condition.

Gradient search is powerful, but its reliance on local information creates two weaknesses. The first is *zigzagging*: on a narrow or sharply curved objective surface (visualized using isoprofit curves, whose tangent is perpendicular to the gradient), the steepest local direction can point across the valley rather than along it, so the algorithm bounces from side to side and makes slow progress. A second-order method such as Newton's method uses curvature information to choose a more informed direction, but calculating and using second-order information can be too expensive for very large models.

The second weakness is *local optimality*: in a non-convex landscape, the algorithm can stop at a local minimum or maximum even when a better global solution exists elsewhere, because at the local optimum every nearby direction looks worse and the gradient offers no escape route. Multi-start and smoothing/temperature-based approaches both improve robustness without eliminating this computational trade-off.

6 | Linear Programming, Duality, and Sensitivity Analysis

6.1 Testing for Linearity

These difficulties motivate linear optimization. A linear function is a sum of terms in which each term contains at most one decision variable, appearing only to the first power; a linear constraint has a linear expression on each side of an equality or inequality. The pricing profit function from the previous chapters is *not* linear, because it multiplies two expressions containing price, ultimately creating a squared term.

To test linearity, separate an expression into terms: constants and constant multiples of a single first-power variable are allowed, while products of decision variables, squares, roots, logarithms, and other nonlinear transformations are not. An equality or inequality is linear only when both sides meet this rule — the connector itself (less than, greater than, or equal to) does not make a constraint nonlinear.

Linear models are deliberately restrictive, but the simplicity is valuable: their behavior is predictable, their feasible regions are convex, and they avoid the local-optimum problem that complicates general nonlinear models. A linear program has a linear objective function and a collection of linear constraints.

6.2 The Wine Cooler and Beer Example

A Montreal company produces wine cooler and beer. Let W be tons of wine cooler and B be tons of beer, with profit measured in thousands of dollars. Wine cooler earns \$3,000 per ton and beer earns \$2,000 per ton. Production consumes two limited raw materials, hops and malt: the company has six tons of hops and eight tons of malt, where each ton of wine cooler uses one unit of hops and two units of malt, and each ton of beer uses two units of hops and one unit of malt.

The manufacturing model can be written as a linear program:

$$\text{Maximize } z = 3W + 2B$$

$$\text{subject to} \quad W + 2B \le 6, \qquad 2W + B \le 8, \qquad W \ge 0, \qquad B \ge 0.$$

The first inequality is the hops capacity and the second is the malt capacity. In matrix form,

$$\text{maximize } c^{\top}x \;\text{ subject to } Ax \le b,\; x \ge 0,$$

where $x = [W, B]^{\top}$, $c = [3, 2]^{\top}$, A has rows $[1, 2]$ and $[2, 1]$, and $b = [6, 8]^{\top}$.

6.3 Graphing the Feasible Region

Each resource boundary can be graphed using intercepts. For the hops equality $W+2B=6$, setting $W=0$ gives $B=3$, and setting $B=0$ gives $W=6$; the malt boundary is constructed the same way. Because usage must be no more than capacity, the feasible side includes the origin, and the feasible polygon is where both resource limits and non-negativity overlap.

The objective's gradient is the constant vector $(3000, 2000)$; only the ratio matters for direction, so it can be viewed as three units toward wine cooler for every two toward beer, and this direction never changes (unlike the quadratic price objective from the previous chapters). Without capacity constraints, profit could increase without bound — the constraints create a finite opportunity set and force the firm to choose a mix.

A linear objective cannot have a uniquely superior point strictly inside a polygon unless the same value extends toward a boundary, so at least one optimum occurs at a vertex. In two dimensions, students can evaluate the objective at each vertex directly; the simplex algorithm generalizes the idea to many dimensions by moving systematically among adjacent basic feasible solutions instead of enumerating every possible point.

6.4 Solving the Linear Program

The feasible vertices are $(0,0)$, $(0,3)$, $(4,0)$, and the intersection of the two resource boundaries. Solving $W+2B=6$ together with $2W+B=8$ gives $W=10/3$ and $B=4/3$. The corresponding objective values (in thousands) are 0, 6, 12, and $38/3$, respectively. Therefore

$$W^\star = \tfrac{10}{3}, \qquad B^\star = \tfrac{4}{3}, \qquad z^\star = \tfrac{38}{3} \approx 12.667 \text{ thousand dollars.}$$

Both resource constraints bind at this solution.

6.5 The Simplex Method

Simplex moves among basic feasible solutions, which correspond geometrically to vertices. At each iteration it selects an entering variable that can improve the objective and a leaving variable determined by feasibility, terminating when no reduced cost indicates further improvement. For a linear program, an optimal solution can always be found at an extreme point (a vertex) of the feasible region: instead of moving through the interior in tiny gradient steps, simplex moves from vertex to vertex, improving the objective until no adjacent vertex is better.

6.6 Sensitivity Analysis and Shadow Prices

Sensitivity analysis (what-if analysis) turns the optimized model into a management tool. An objective coefficient may change if a product's profit margin changes; a constraint coefficient may change if production becomes more or less resource efficient; a right-hand side may change if the company acquires more hops or malt. The analyst asks whether

the same production plan remains optimal, how much profit changes, and which resource is most valuable at the margin.

A *shadow price* is the marginal improvement in the optimal objective associated with one additional unit of a binding resource, within a relevant range; a zero shadow price often indicates that the resource is not currently scarce. Dual variables formalize these marginal values — managers are often more interested in the value of extra capacity, or the risk of changing assumptions, than in a single static optimum.

6.7 The Dual Problem

How can we obtain shadow prices? The short answer: every linear optimization problem has a corresponding linear optimization problem called its *dual problem*, which can be constructed mechanically from the original (*primal*) problem. By solving the dual problem, we obtain the shadow price associated with each constraint.

The dual problem associated with the wine cooler and beer example above is

$$\text{Minimize } 6y_1 + 8y_2 \quad \text{subject to} \quad y_1 + 2y_2 \geq 3, \quad 2y_1 + y_2 \geq 2, \quad y_1, y_2 \geq 0.$$

How do we arrive at this dual problem? Imagine that, instead of using our raw materials ourselves, we consider selling our raw material capacity at prices y_1 (per unit of hops) and y_2 (per unit of malt). The question is: what would be a *fair* price to charge?

To answer this, we first need these prices to be *incentive compatible*. That is, the prices must be high enough that we would rather sell the raw materials outright than use them to manufacture and sell wine coolers or beer ourselves. Since each unit of wine cooler requires 1 unit of hops and 2 units of malt, at prices y_1 and y_2 the implied raw-material cost of producing one unit of wine cooler is $y_1 + 2y_2$. Incentive compatibility requires that this cost be at least as large as the marginal profit of \$3 earned per unit of wine cooler sold; otherwise, we would prefer to manufacture wine coolers rather than sell the raw materials. This gives the incentive compatibility constraint for wine cooler:

$$y_1 + 2y_2 \geq 3.$$

Applying the same logic to beer, which requires 2 units of hops and 1 unit of malt per unit produced, with a marginal profit of \$2, yields the incentive compatibility constraint for beer:

$$2y_1 + y_2 \geq 2.$$

Finally, since selling raw materials at a negative price would never make sense, the prices must satisfy the non-negativity constraints:

$$y_1, y_2 \geq 0.$$

Given these prices, and given that we have 6 units of hops and 8 units of malt available, selling off our entire raw material capacity would generate revenue

$$6y_1 + 8y_2.$$

A fair price is one that keeps this revenue as low as possible while still satisfying the incentive compatibility constraints above – any lower, and the prices would fail to be

incentive compatible; any higher, and we would be overpaying ourselves relative to what the raw materials are actually worth in production. This reasoning leads to the dual problem:

$$\text{Minimize } 6y_1 + 8y_2 \quad \text{subject to} \quad y_1 + 2y_2 \geq 3, \quad 2y_1 + y_2 \geq 2, \quad y_1, y_2 \geq 0.$$

Remarkably, solving the dual problem yields prices y_1 and y_2 whose resulting payoff is *exactly equal* to the optimal objective value of the original primal problem. This equivalence is known as the *duality principle* for linear optimization problems. In our example, solving the dual problem gives

$$y_1 = \frac{1}{3}, \qquad y_2 = \frac{4}{3} \quad \text{(in thousands of dollars)},$$

with dual objective value $6\left(\frac{1}{3}\right) + 8\left(\frac{4}{3}\right) = \frac{38}{3}$, exactly matching the optimal objective value of the primal problem – illustrating the duality principle.

These dual values also have a natural economic interpretation as *shadow prices*: one additional ton of hops is worth about \$0.333 thousand, while one additional ton of malt is worth about \$1.333 thousand, *provided the current optimal basis remains unchanged.*

6.8 Allowable Increase and Allowable Decrease for Constraints

The shadow price $y_1 = 1/3$ for hops and $y_2 = 4/3$ for malt tells us the *rate* at which the optimal profit changes as we relax or tighten a resource constraint. However, this rate is only valid over a limited range of the right-hand side (RHS) value, because the shadow price is tied to a specific *optimal basis* – that is, a specific set of binding constraints and basic (nonzero) decision variables. Once the RHS changes enough that the optimal basis would change (e.g., a decision variable that was positive would be forced to become negative, or a new constraint would become binding), the shadow price itself changes. The *allowable increase* and *allowable decrease* tell us exactly how far we can change a given RHS value while keeping the current basis – and hence the current shadow price – valid.

General Principle

Recall that at the optimal solution, wine cooler and beer production levels are

$$x_1 = \frac{10}{3}, \qquad x_2 = \frac{4}{3},$$

with both the hops and malt constraints binding. Since both x_1 and x_2 are positive (basic), the current basis consists of these two variables, and it is determined by the following *basis matrix*, formed from their coefficients in the two binding constraints:

$$B = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}, \qquad B^{-1} = \begin{bmatrix} -1/3 & 2/3 \\ 2/3 & -1/3 \end{bmatrix}.$$

If $b = (b_1, b_2)^\top$ denotes the RHS vector (available hops and malt), then the values of the basic variables are recovered as

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = B^{-1} b.$$

The current basis remains optimal as long as this solution stays *feasible*, i.e., as long as $x_1 \geq 0$ and $x_2 \geq 0$. This feasibility requirement is precisely what determines the allowable range for each RHS.

Ranging the Hops Constraint ($b_1 = 6$)

Suppose the hops availability changes from 6 to $6 + \Delta_1$, while malt availability stays fixed at 8. The new basic solution is

$$\begin{pmatrix} x_1(\Delta_1) \\ x_2(\Delta_1) \end{pmatrix} = B^{-1} \begin{pmatrix} 6 + \Delta_1 \\ 8 \end{pmatrix} = \begin{pmatrix} 10/3 \\ 4/3 \end{pmatrix} + \Delta_1 \begin{pmatrix} -1/3 \\ 2/3 \end{pmatrix}.$$

Requiring both entries to remain nonnegative gives

$$x_1(\Delta_1) = \frac{10}{3} - \frac{\Delta_1}{3} \geq 0 \implies \Delta_1 \leq 10,$$

$$x_2(\Delta_1) = \frac{4}{3} + \frac{2\Delta_1}{3} \geq 0 \implies \Delta_1 \geq -2.$$

Hence $-2 \leq \Delta_1 \leq 10$, so the hops availability can range from $6 - 2 = 4$ to $6 + 10 = 16$ tons without changing the optimal basis. In other words:

$$\text{Allowable increase for hops} = 10, \qquad \text{Allowable decrease for hops} = 2.$$

Within this range, the shadow price $y_1 = 1/3$ (thousand dollars per ton) remains exactly correct.

Ranging the Malt Constraint ($b_2 = 8$)

Similarly, suppose malt availability changes from 8 to $8 + \Delta_2$, with hops fixed at 6:

$$\begin{pmatrix} x_1(\Delta_2) \\ x_2(\Delta_2) \end{pmatrix} = \begin{pmatrix} 10/3 \\ 4/3 \end{pmatrix} + \Delta_2 \begin{pmatrix} 2/3 \\ -1/3 \end{pmatrix}.$$

Requiring nonnegativity:

$$x_1(\Delta_2) = \frac{10}{3} + \frac{2\Delta_2}{3} \geq 0 \implies \Delta_2 \geq -5,$$

$$x_2(\Delta_2) = \frac{4}{3} - \frac{\Delta_2}{3} \geq 0 \implies \Delta_2 \leq 4.$$

Hence $-5 \leq \Delta_2 \leq 4$, so malt availability can range from $8 - 5 = 3$ to $8 + 4 = 12$ tons while the current basis – and the shadow price $y_2 = 4/3$ – remain valid:

$$\text{Allowable increase for malt} = 4, \qquad \text{Allowable decrease for malt} = 5.$$

Summary

Resource	Shadow Price	Allowable Decrease	Allowable Increase
Hops ($b_1 = 6$)	1/3	2	10
Malt ($b_2 = 8$)	4/3	5	4

These ranges tell us, for instance, that acquiring up to 10 additional tons of hops would each be worth \$0.333 thousand, but beyond that point the current basis would no longer be optimal – production would shift enough that a new set of binding constraints (and a new shadow price) would take over. The same logic applies symmetrically to decreases in availability.

6.9 Allowable Increase and Allowable Decrease for Objective Coefficients

Just as shadow prices are tied to a specific optimal *primal* basis, they are equally tied to a specific optimal *dual* solution. When an objective coefficient changes, the values of the decision variables (x_1, x_2) do not change at all – what changes is whether the *current dual basis remains optimal*, which now hinges on whether the associated dual prices y_1, y_2 remain *dual feasible* (i.e., remain nonnegative). The allowable increase and allowable decrease for an objective coefficient tell us how far that coefficient can move while the current dual prices – and hence the current optimal production plan – remain valid.

General Principle

Recall that x_1 (wine cooler) and x_2 (beer) are both basic (positive) in the optimal solution, so both resource constraints are binding. This means the dual prices are recovered from the objective coefficients of the basic variables, $c_B = (c_1, c_2)$, via

$$y^\top = c_B^\top B^{-1}, \qquad \text{where } B^{-1} = \begin{bmatrix} -1/3 & 2/3 \\ 2/3 & -1/3 \end{bmatrix}.$$

As long as the basis itself does not change, this formula tells us exactly how y_1 and y_2 move as we perturb c_1 or c_2. The current basis remains optimal precisely as long as the resulting y_1 and y_2 stay *nonnegative* – this is the dual feasibility requirement. The moment a dual price would be forced negative, the current basis is no longer optimal, and production would shift to a different combination of products.

Ranging the Wine Cooler Coefficient ($c_1 = 3$)

Suppose the marginal profit of wine cooler changes from 3 to $3 + \Delta_1$, holding $c_2 = 2$ fixed. The resulting dual prices are

$$y_1(\Delta_1) = (3+\Delta_1)\left(-\frac{1}{3}\right)+2\left(\frac{2}{3}\right) = \frac{1}{3}-\frac{\Delta_1}{3}, \qquad y_2(\Delta_1) = (3+\Delta_1)\left(\frac{2}{3}\right)+2\left(-\frac{1}{3}\right) = \frac{4}{3}+\frac{2\Delta_1}{3}.$$

Requiring both prices to remain nonnegative:

$$y_1(\Delta_1) \geq 0 \implies \Delta_1 \leq 1, \qquad y_2(\Delta_1) \geq 0 \implies \Delta_1 \geq -2.$$

Hence $-2 \leq \Delta_1 \leq 1$, so the wine cooler profit margin can range from $3 - 2 = 1$ to $3 + 1 = 4$ without changing the optimal production plan:

$$\text{Allowable increase for } c_1 = 1, \qquad \text{Allowable decrease for } c_1 = 2.$$

Within this range, it remains optimal to produce $x_1 = 10/3$ wine coolers and $x_2 = 4/3$ beers; only the dual prices y_1, y_2 shift as c_1 moves, until one of them hits zero at the boundary of the range.

Ranging the Beer Coefficient ($c_2 = 2$)

Similarly, suppose the marginal profit of beer changes from 2 to $2 + \Delta_2$, holding $c_1 = 3$ fixed:

$$y_1(\Delta_2) = 3\left(-\frac{1}{3}\right) + (2+\Delta_2)\left(\frac{2}{3}\right) = \frac{1}{3} + \frac{2\Delta_2}{3}, \qquad y_2(\Delta_2) = 3\left(\frac{2}{3}\right) + (2+\Delta_2)\left(-\frac{1}{3}\right) = \frac{4}{3} - \frac{\Delta_2}{3}.$$

Requiring nonnegativity:

$$y_1(\Delta_2) \geq 0 \implies \Delta_2 \geq -\frac{1}{2}, \qquad y_2(\Delta_2) \geq 0 \implies \Delta_2 \leq 4.$$

Hence $-1/2 \leq \Delta_2 \leq 4$, so the beer profit margin can range from $2 - 0.5 = 1.5$ to $2 + 4 = 6$ while the current production plan remains optimal:

$$\text{Allowable increase for } c_2 = 4, \qquad \text{Allowable decrease for } c_2 = 0.5.$$

Summary

Product	Current Coefficient	Allowable Decrease	Allowable Increase
Wine cooler (c_1)	3	2	1
Beer (c_2)	2	0.5	4

Intuitively, these ranges tell us how sensitive the production plan is to uncertainty or negotiation in profit margins. For example, the profit margin on wine cooler could drop as low as \$1 thousand per unit, or rise as high as \$4 thousand per unit, without it becoming optimal to change how many wine coolers or beers we produce. Beyond these limits, one of the dual prices would be driven negative, signaling that the current basis – and therefore the current production plan – is no longer optimal, and a new set of basic variables would take over.

6.10 Reduced Cost as the Dual Price of the Non-Negativity Constraint

So far, we have treated the non-negativity requirements $x_1 \geq 0$ and $x_2 \geq 0$ as an afterthought – a technical restriction rather than a genuine constraint. In fact, each non-negativity constraint has its own dual price, exactly like the hops and malt constraints do. This dual price is called the *reduced cost* of the corresponding decision variable, and it tells us how much the objective coefficient of that variable would need to improve before it would become profitable to produce a positive amount of it.

General Principle

Rewrite each non-negativity constraint as $-x_j \leq 0$, or equivalently think of $x_j \geq 0$ as its own constraint with an associated dual price $z_j \geq 0$ (for a maximization problem). By complementary slackness, exactly one of the following holds for each decision variable x_j:

- $x_j > 0$ (the variable is *basic*), in which case its non-negativity constraint is *not* binding, so its dual price – its reduced cost – must be *zero*;
- $x_j = 0$ (the variable is *nonbasic*), in which case its non-negativity constraint *is* binding, and its reduced cost is generally nonzero.

Formally, the reduced cost of variable x_j is computed the same way we priced out the wine cooler and beer constraints earlier: it is the marginal profit of x_j minus the implied cost of the resources it consumes, valued at the shadow prices y_1, y_2:

$$\bar{c}_j = c_j - (a_{1j}y_1 + a_{2j}y_2)\,,$$

where a_{1j} and a_{2j} are the amounts of hops and malt required per unit of x_j. At optimality (for a maximization problem), we must have $\bar{c}_j \leq 0$ for every variable – otherwise, it would still be profitable to increase x_j, and the solution would not be optimal.

Reduced Costs for Wine Cooler and Beer

In our example, both $x_1 = 10/3$ and $x_2 = 4/3$ are strictly positive, so both are basic variables. Their non-negativity constraints are not binding, and therefore:

$$\bar{c}_1 = 0, \qquad \bar{c}_2 = 0.$$

This is confirmed directly: since $y_1 = 1/3$ and $y_2 = 4/3$ were constructed exactly so that $y_1 + 2y_2 = 3 = c_1$ and $2y_1 + y_2 = 2 = c_2$ hold with equality, the reduced costs of x_1 and x_2 are precisely zero. In general, *every basic variable has a reduced cost of zero* – this is precisely why its incentive compatibility constraint in the dual problem is binding.

Illustrating a Nonzero Reduced Cost

To see when a reduced cost is nonzero, suppose the brewery considered a third product, say *cider*, requiring 3 units of hops and 3 units of malt per unit produced, with a marginal profit of $c_3 = 2$ (in thousands of dollars). Even without solving a new optimization problem, we can price out cider using the existing shadow prices $y_1 = 1/3$ and $y_2 = 4/3$:

$$\bar{c}_3 = c_3 - (3y_1 + 3y_2) = 2 - \left(3 \cdot \frac{1}{3} + 3 \cdot \frac{4}{3}\right) = 2 - 5 = -3.$$

The reduced cost of cider is -3, meaning that producing one unit of cider would use up hops and malt worth \$5 thousand (at their shadow prices) while generating only \$2 thousand in profit – a net loss of \$3 thousand per unit. This confirms that cider should *not* be produced at current prices, consistent with $x_3 = 0$ in the optimal solution. Moreover, the reduced cost tells us exactly how much the profit margin on cider would need to increase before it becomes worth producing: it would need to rise by at least 3, from 2 to 5, before cider becomes attractive at the margin.

Interpretation

The reduced cost therefore has a clean economic meaning: it is the answer to the question, *"If I were forced to produce at least one unit of this product, how much better or worse off would I be, relative to just using its resources elsewhere?"* A reduced cost of zero means the variable is already being produced at a profitable level (or is indifferent to entering); a negative reduced cost (in a maximization problem) means the variable is correctly excluded from the optimal solution, and quantifies exactly how much its profitability would need to improve to change that conclusion.

6.11 Automating Sensitivity Analysis in Practice

The calculations above were carried out by hand to illustrate the underlying *principles* of duality and sensitivity analysis – namely, that shadow prices and allowable ranges come from keeping the optimal primal basis (for RHS ranging) or the optimal dual prices (for objective coefficient ranging) unchanged. In real-world applications, however, we almost never compute these values manually. Both Excel and Python can generate shadow prices, allowable increases, and allowable decreases automatically, with no manual linear algebra required.

Excel: Sensitivity Report

In Excel, after solving a linear optimization problem using **Solver**, we can request a **Sensitivity Report** directly from the Solver Results dialog box (by selecting "Sensitivity" before clicking OK). This report automatically provides:

- the shadow price for each constraint, along with the allowable increase and allowable decrease in its RHS value over which that shadow price remains valid;
- the allowable increase and allowable decrease for each objective coefficient over which the current optimal solution remains unchanged.

No manual computation of B^{-1} or dual feasibility conditions is required – Excel performs all of this internally and simply reports the final ranges.

Python: Sensitivity Analysis with `scipy` or Solver APIs

In Python, sensitivity analysis can similarly be obtained automatically after solving a linear program. For example, using `scipy.optimize.linprog` with the `'highs'` method, the returned solution object includes marginal values (shadow prices) for constraints. Other solvers, such as `PuLP`, `Gurobi`, or `CPLEX`, provide even more complete sensitivity reports through built-in attributes, for example:

- `constraint.pi` or `constraint.Pi` – the shadow price (dual value) of a constraint;
- `constraint.SARHSLow` / `constraint.SARHSUp` – the allowable range for a constraint's RHS;
- `variable.SAObjLow` / `variable.SAObjUp` – the allowable range for an objective coefficient.

A few lines of code applied to the model are sufficient to generate the same shadow prices and allowable increase/decrease values that we derived by hand above.

Why the Principles Still Matter

Given that Excel and Python can generate these reports instantly, it is natural to ask why we bother deriving them by hand at all. The answer is that software can tell us *what* the shadow prices and allowable ranges are, but only an understanding of duality and basis feasibility tells us *why* they take those values, and more importantly, when they can and cannot be trusted. Knowing the underlying principles allows us to:

- correctly interpret a sensitivity report rather than misapply it (e.g., recognizing that ranges assume *one* coefficient or RHS changes at a time, holding all others fixed);
- anticipate when a proposed change (e.g., a large increase in raw material availability) falls *outside* the allowable range, signaling that the shadow price no longer applies and the problem should be re-solved;
- explain and justify sensitivity results to others – for instance, explaining to a manager *why* paying more than a certain shadow price for extra hops is not worthwhile.

In short, software automates the arithmetic, but the underlying duality principles remain essential for using and interpreting the results correctly in practice.

6.12 Key Takeaways from Module 1

Across the session, four ideas carry the most weight: optimization is the decision engine behind both operational planning and machine-learning training; good formulation requires decision variables, an objective function, and constraints that faithfully represent the real situation; gradient search is scalable because it repeatedly uses inexpensive local information, but step size, curvature, feasibility, and local optima all matter; and linear programming trades modeling flexibility for exceptional structure, reliability, and interpretability.

- Optimization underlies operational decisions and the training of machine-learning models.
- Every optimization model needs decision variables, an objective function, and constraints.
- Convexity makes optimization more predictable; non-convex problems can contain difficult local optima.
- Gradient search uses a local slope for direction and a step size for distance.
- Small steps can be slow; large steps can overshoot or zigzag. Adaptive methods adjust the step size.
- Linear programs use a linear objective and linear constraints, creating a structured convex feasible region.

- The simplex method searches extreme points of a linear program rather than taking tiny interior steps.
- Sensitivity analysis studies how the optimal solution changes when coefficients or capacities change.
- The course emphasizes conceptual modeling and interpretation, with Python or Excel used for implementation.

For study purposes, students should be able to identify the three components of an optimization problem, distinguish scalar and vector decisions, explain convexity in plain language, describe how gradient and step size play different roles, recognize why gradient search can zigzag or get stuck, test whether a function is linear, formulate the manufacturing objective and capacity constraints, and explain why simplex focuses on extreme points.

Part II

Probability, Bayes' Rule, Decision Trees, and the Value of Information (Module 2)

7 | Probability Models, Sample Spaces, Events, and Counting

This chapter covers probability models, sample spaces, events, counting, and how these ideas connect to language models and generative AI.

7.1 Defining a Probability

Probability is defined as a normalized number measuring how likely an event is. A probability of zero means impossibility within the model, one means certainty, and intermediate values quantify uncertainty.

$$0 \leq P(A) \leq 1.$$

7.2 Sample Spaces and Events

A probability model has three conceptual ingredients: an experiment, a sample space, and probabilities assigned to outcomes. The sample space, usually written Ω, contains every elementary outcome that the model recognizes. An event A is a subset of Ω and may contain one or many elementary outcomes.

$$A \subseteq \Omega, \qquad \sum_{\omega \in \Omega} P(\omega) = 1.$$

The second axiom is the normalization rule: all elementary-outcome probabilities must sum to one. Increasing the probability of one outcome therefore requires decreasing probability elsewhere. A softmax layer in a neural network is mentioned as a standard way to convert arbitrary scores into positive numbers that sum to one.

7.3 Large Language Models as a Motivating Example

Large language models provide the motivating example. A token is treated as an elementary outcome, the vocabulary is the sample space for the next-token decision, and the model estimates a probability distribution over that vocabulary. A marginal probability such as $P(\text{school})$ ignores context. A language model instead estimates a conditional probability such as the probability that the next token is "school" given the preceding prompt.

7.4 Simple Experiments and Structured Sample Spaces

Simple experiments make the same structure visible. One die has six elementary outcomes. Two dice have thirty-six ordered outcomes. A quality test may have success and failure. A digital image has an enormous structured state space determined by pixel positions and red, green, and blue intensities. Generative modeling is difficult largely because real sample spaces are huge and their probabilities are not uniform.

7.5 Counting With and Without Replacement

For two sequential draws from three colors with replacement, the ordered sample space has three times three, or nine, outcomes. If all colors and draws are symmetric, each outcome has probability one ninth. Without replacement, only six ordered color pairs remain when each color is initially represented once. The lecture uses this distinction to show that counting depends on the experimental protocol.

7.6 Combining Events: Union and Intersection

An event can combine elementary outcomes. The event of drawing the same color twice contains red-red, green-green, and yellow-yellow.

$$P(\text{same color}) = \frac{1}{9} + \frac{1}{9} + \frac{1}{9} = \frac{1}{3}.$$

This calculation uses additivity for mutually exclusive outcomes. More generally, the union of two events needs a correction when they overlap.

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

In the marble example, let A mean at least one red and B mean two marbles of the same color. $P(A)$ is five ninths, $P(B)$ is three ninths, and their intersection is red-red with probability one ninth. Therefore the union has probability seven ninths. Simply adding five ninths and three ninths would double-count red-red.

7.7 The Complement Rule

The complement rule is another consequence of normalization.

$$P(A^c) = 1 - P(A).$$

The practical discipline is to define the experiment before calculating. Specify whether order matters, whether sampling uses replacement, whether outcomes are equally likely, and exactly which elementary outcomes constitute the event. Most probability errors begin with an ambiguous sample space rather than difficult arithmetic.

7.8 Elementary Outcomes versus Events

The distinction between an elementary outcome and an event matters computationally. An elementary outcome is indivisible at the model's chosen resolution. An event is any collection of elementary outcomes. The same real-world description can produce different sample spaces depending on the question. If the experiment is drawing one marble and recording color, the outcomes are red, green, and yellow. If the question is the number of red marbles in two draws, the outcomes are zero, one, and two, even though the underlying ordered color pairs are more detailed.

7.9 Counting Rules Under Uniformity

Counting rules supply probabilities only when symmetry assumptions are justified. If all elementary outcomes are equally likely, the probability of event A is its number of favorable outcomes divided by the number of outcomes in Ω.

$$P(A) = \frac{|A|}{|\Omega|} \qquad \text{(under uniformity)}.$$

With replacement, two draws from three colors create nine ordered pairs because the first and second positions each have three possibilities. Without replacement, the count depends on how many physical marbles of each color exist. The simplified three-times-two count assumes one marble per color and ordered observations. This illustrates why analysts must not use a counting formula before specifying inventory, replacement, and order.

7.10 Joint-Probability Tables

A joint-probability table is a useful representation for two categorical variables. Rows encode the first draw and columns encode the second. Each cell is an elementary joint outcome. Row sums and column sums are marginal probabilities, and selected blocks or diagonals represent events. The table makes overlap visible and helps prevent double counting.

7.11 Countable Additivity and Inclusion-Exclusion

For disjoint events A_i, countable additivity gives:

$$P\left(\bigcup_i A_i\right) = \sum_i P(A_i), \qquad \text{when the } A_i \text{ are pairwise disjoint.}$$

The general two-event addition rule follows by splitting $A \cup B$ into three disjoint regions: only A, only B, and the intersection. The subtraction term removes the duplicate intersection. For three or more overlapping events, inclusion-exclusion alternates sums and subtractions of intersections.

7.12 Empirical Estimation from Data

Uniform distributions are useful initial models, but observed systems are rarely uniform. A biased die, unequal marble counts, language frequencies, and image structure all produce unequal probabilities. Empirical estimation uses relative frequencies from data.

$$\hat{P}(A) = \frac{\text{number of observed cases in } A}{\text{total observations}}.$$

As data accumulate, the estimate can be updated. A generative model performs a far more structured version of this task, sharing statistical strength across contexts rather than independently counting every possible sentence or image.

7.13 Softmax as a Normalization Mechanism

Softmax is the normalization mechanism briefly referenced in the lecture. Given model scores z_1 through z_K, it defines:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}.$$

Every output is positive and the outputs sum to one. Softmax does not guarantee that probabilities are well calibrated or correct; it guarantees only the mathematical form of a categorical probability distribution. Training data and the loss function determine whether those probabilities become useful.

8 | Conditional Probability, Bayes' Rule, and Independence

8.1 Defining Conditional Probability

Conditional probability restricts attention to cases in which another event is already known to have occurred. It is written $P(A \mid B)$. The event B becomes the new reference population, so the intersection of A and B must be normalized by $P(B)$.

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \qquad P(B) > 0.$$

For a fair die, let A be a result less than four, so $A = \{1, 2, 3\}$. Let B be an odd result, so $B = \{1, 3, 5\}$. The intersection is $\{1, 3\}$. Therefore:

$$P(A \mid B) = \frac{2/6}{3/6} = \frac{2}{3}.$$

The complement within the same condition is five, so $P(A^c \mid B)$ equals one third. The two conditional probabilities add to one. This illustrates why the denominator is required: it renormalizes probabilities after the sample space is restricted to B.

8.2 The Mask-and-COVID Example

The mask-and-COVID table provides a practical example. Fifteen percent of the population both wears masks and catches COVID. Forty-five percent wears masks without catching COVID. Another fifteen percent catches COVID without wearing masks, leaving twenty-five percent in neither group. The marginal COVID probability is thirty percent and the mask probability is sixty percent.

$$P(\text{COVID}) = 0.15 + 0.15 = 0.30.$$

$$P(\text{COVID} \mid \text{mask}) = \frac{0.15}{0.60} = 0.25.$$

Within this illustrative table, observing mask use changes the estimated COVID probability from thirty percent to twenty-five percent, a five-percentage-point reduction. The thirty percent value is described as a prior or marginal probability; twenty-five percent is the posterior after conditioning on mask use. The table is a teaching example, not evidence for a real-world causal estimate.

8.3 The Multiplication Rule and Bayes' Rule

Rearranging conditional probability yields the multiplication rule.

$$P(A \cap B) = P(B)P(A \mid B) = P(A)P(B \mid A).$$

Equating the two versions and solving for $P(A \mid B)$ produces Bayes' rule.

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}.$$

Bayes' rule has two interpretations. It reverses the direction of conditioning, allowing $P(A \mid B)$ to be derived from $P(B \mid A)$. It also updates a prior belief $P(A)$ to a posterior belief $P(A \mid B)$ using the likelihood $P(B \mid A)$ and the evidence $P(B)$. The denominator ensures that posterior probabilities across competing hypotheses sum to one.

For the mask example, $P(\text{mask} \mid \text{COVID})$ is one half because half of the COVID group wears masks. Bayes' rule gives:

$$P(\text{COVID} \mid \text{mask}) = \frac{0.50 \times 0.30}{0.60} = 0.25.$$

8.4 Independence

Independence means that observing B does not alter the probability of A.

$$A \text{ and } B \text{ are independent if } P(A \mid B) = P(A).$$

An equivalent multiplication test is:

$$P(A \cap B) = P(A)P(B).$$

Independence is symmetric. If A is independent of B, then B is independent of A. It should not be confused with mutual exclusivity. Two mutually exclusive events with positive probability cannot be independent because occurrence of one makes the other impossible.

8.5 Connections to Generative AI

The lecture connects these formulas to generative AI. Each additional prompt token changes the conditioning information, and the model recomputes a distribution over possible next tokens. Search autocomplete is a simpler example. The mathematical core is repeated conditional-probability estimation under a very large and structured sample space.

Conditional probability can be visualized as filtering and rescaling. First discard outcomes outside B. Then retain the portion of B that also belongs to A. Finally divide by the total

mass of B. This procedure explains both the numerator intersection and denominator normalization without relying on memorization.

8.6 The Law of Total Probability

The law of total probability reconstructs a marginal probability from conditional pieces. If B_1 through B_k form a mutually exclusive and exhaustive partition, then:

$$P(A) = \sum_{i=1}^{k} P(A \mid B_i)P(B_i).$$

This formula supplies the denominator in many Bayes calculations. In a diagnostic setting, A might be a positive test and the B_i might be disease states. In the economic-signal example later in the lecture, A is a positive forecast and the partition is good versus bad economy.

8.7 Base Rates and Posterior Odds

Bayes' rule is most informative when base rates matter. A high $P(B \mid A)$ does not automatically imply a high $P(A \mid B)$. The posterior combines the likelihood with the prior prevalence and divides by the overall evidence. Ignoring $P(A)$ is the base-rate fallacy.

Posterior odds provide an equivalent technical form.

$$\text{Posterior odds of } A \text{ vs. } A^c = \text{prior odds} \times \text{likelihood ratio}.$$

$$\text{Likelihood ratio} = \frac{P(B \mid A)}{P(B \mid A^c)}.$$

This form separates prior belief from the evidential strength of B. A likelihood ratio greater than one favors A; below one favors A^c; equal to one carries no information about A.

In the mask table, the posterior comparison is descriptive. Conditioning identifies association in the table, not causation. A causal conclusion would require stronger design assumptions about confounders, selection, measurement, and how mask use was assigned. The lecture's mathematical point is how subgroup information changes probability.

8.8 Conditional Independence

Independence can also be checked through complements and factorization. If A and B are independent, A is independent of B^c, A^c is independent of B, and their joint table factors into row and column marginals. Zero correlation is generally weaker than independence for numeric variables, although the two coincide under special distributional assumptions.

Conditional independence is central to larger models. Events A and B may be dependent marginally but independent after conditioning on C.

$$A \perp B \mid C \quad \Longleftrightarrow \quad P(A \mid B, C) = P(A \mid C).$$

Graphical models, naive Bayes, hidden Markov models, and many sequence models exploit conditional-independence structure to avoid estimating an impossibly large unrestricted joint distribution.

8.9 The Chain Rule for Language Generation

For language generation, the chain rule decomposes a sentence probability into next-token conditionals.

$$P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1}).$$

A language model estimates these factors. Generating text then samples or selects from each next-token distribution and appends the chosen token to the conditioning context. The probability module therefore connects directly to the computational behavior of generative systems.

9 | Decision Trees, Expected Value, and Backward Induction

9.1 From Probability to Sequential Decisions

The second half of this part applies probability to sequential decisions. A decision tree separates controllable choices from uncontrollable states. A square represents a decision node. A circle represents a chance or event node. Branches from a decision node are available actions. Branches from a chance node are possible states with probabilities. Terminal nodes carry final payoffs.

9.2 The Real-Estate Example

The real-estate example has three initial choices: build an apartment, office, or warehouse. The economy is then good with probability 0.6 or bad with probability 0.4. Payoffs are in millions of dollars. Apartment pays fifty in a good economy and thirty in a bad economy. Office pays one hundred in good and minus forty in bad. Warehouse pays thirty in good and ten in bad.

9.3 Expected Monetary Value

Expected monetary value is the probability-weighted payoff.

$$\mathrm{EMV}(a) = \sum_{s} P(s \mid a)\,\mathrm{payoff}(a, s).$$

With common state probabilities, the three calculations are:

$$\mathrm{EMV}(\text{apartment}) = 0.6(50) + 0.4(30) = 42.$$

$$\mathrm{EMV}(\text{office}) = 0.6(100) + 0.4(-40) = 44.$$

$$\mathrm{EMV}(\text{warehouse}) = 0.6(30) + 0.4(10) = 22.$$

A risk-neutral decision maker chooses office because forty-four is the largest EMV, despite the negative payoff in a bad economy. This criterion evaluates average monetary performance over repeated comparable decisions. A risk-averse decision maker could

prefer a different alternative if utility, downside constraints, or risk measures replaced raw monetary value.

9.4 Backward Induction (Rollback)

Decision trees are solved by rollback, also called folding back or backward induction. Begin at the nodes closest to the terminal payoffs and work toward the root. At a chance node, calculate the expected value of its children. At a maximizing decision node, retain the child with the largest value. For cost-minimization trees, retain the smallest value instead.

$$V(\text{chance node}) = \sum_i p_i V_i.$$

$$V(\text{decision node}) = \max_{i \text{ feasible}} V_i \qquad \text{(payoff maximization)}.$$

Suboptimal branches are pruned. A branch marked with selection probability one identifies the chosen action; branches marked zero are not selected under the optimal deterministic policy. If two actions tie exactly, either can be selected without changing value, or a randomized policy can mix between them.

9.5 Software Support and Its Limits

The lecture demonstrates SilverDecisions, an open-source browser tool. A user creates square and circular nodes, attaches probabilities and payoffs, and lets the software roll the tree back automatically. A special placeholder can calculate the missing probability on a chance node so that outgoing branch probabilities sum to one. If they do not sum to one, the tool warns the user.

For nested chance nodes, rollback still works one layer at a time. First evaluate the deepest chance node. Treat its calculated expected value as the payoff passed to its parent. Continue until the root is reached. This recursive structure underlies dynamic programming, reinforcement learning, game-playing systems, and other multistage decision models.

9.6 Timing and the Bellman Recursion

A decision tree encodes timing as well as outcomes. Moving a chance node before a decision node means the information becomes available before the decision. This change in order is the basis for calculating the value of information.

A formal decision tree alternates information states and actions. Let $V(n)$ denote the value of node n. At a terminal node, V equals the stated payoff. At a chance node, V is a probability-weighted average of child values. At a decision node, V is the best available child value under the chosen objective. This recursion is the Bellman principle in a simple finite tree.

$$V(n) = \begin{cases} \text{payoff}(n) & \text{at a terminal node,} \\ \sum_i p_i \, V(\text{child}_i) & \text{at a chance node,} \\ \max_i V(\text{child}_i) & \text{at a decision node.} \end{cases}$$

The rollback order matters because a decision can depend only on information observed before that node. Solving forward by choosing the branch with the largest immediate payoff can fail when later risks or rewards differ. Backward induction summarizes all downstream consequences before the earlier choice is evaluated.

9.7 Risk Aversion and Expected Utility

The real-estate example assumes risk neutrality. Expected monetary value is linear in money, so a gain of ten offsets an equally likely loss of ten. Under expected utility theory, a risk-averse decision maker applies a concave utility function u before averaging.

$$\mathbb{E}[u(a)] = \sum_s P(s) \, u(\text{payoff}(a, s)).$$

The optimal action maximizes expected utility, not necessarily expected money. Decision trees can also include constraints, risk measures, or multi-attribute scores when monetary expectation alone is inappropriate.

9.8 A Nested Chance Node Example

The nested-chance-node classroom question illustrates recursion. Suppose a deeper chance node has value forty-two. Its parent chance node has branches paying fifty with probability 0.6, the nested value forty-two with probability 0.2, and thirty with probability 0.2.

$$\text{Parent value} = 0.6(50) + 0.2(42) + 0.2(30) = 44.4.$$

The inner uncertainty is solved once and replaced by its expected continuation value. This compression makes large trees manageable conceptually, though their number of states may still grow exponentially.

9.9 Pruning and Ties

Pruning records the policy. At a decision node, rejected actions can be visually crossed out because they will not be used under the assumed model. At a chance node, branches are never pruned merely because their payoff is unattractive; chance outcomes remain possible and must be averaged.

When two actions tie, there are multiple optimal deterministic policies. Any mixture between tied actions has the same expected value in the basic linear model. Randomization becomes strategically important in adversarial games because predictable actions can be exploited. The lecture briefly connects this idea to game theory and generative adversarial networks, though those topics lie beyond the module.

9.10 Auditing a Decision Tree

SilverDecisions supports the mechanics but does not validate the model's assumptions. Users must still verify timing, branch labels, probability normalization, payoff units, and whether a node is genuinely a decision or chance. Copying a subtree is efficient only when its structure is truly shared; branch-specific probabilities and payoffs must be edited afterward.

A reliable audit proceeds from right to left: confirm every terminal payoff, confirm that outgoing chance probabilities sum to one, recompute each chance-node expected value, verify each decision-node maximum or minimum, and compare the selected path with the business narrative.

10 | Perfect and Imperfect Information

10.1 The Uninformed Benchmark

Without additional information, the real-estate decision has value forty-four and the optimal action is office. Perfect information means the true economic state is revealed before the building decision. The tree therefore starts with the good-or-bad chance node, followed by a decision node under each realized state.

10.2 Expected Value of Perfect Information

If the economy is known to be good, the best payoff is one hundred from office. If it is known to be bad, the best payoff is thirty from apartment. The expected value with perfect information is:

$$\mathrm{EV}_{\mathrm{perfect}} = 0.6(100) + 0.4(30) = 72.$$

The expected value of perfect information is the improvement over the best uninformed decision.

$$\mathrm{EVPI} = 72 - 44 = 28.$$

Twenty-eight million dollars is an upper bound on what a decision maker should pay for any information about the economy. Real forecasts are imperfect, so their value cannot exceed EVPI when evaluated under the same assumptions.

10.3 Setting Up the Imperfect-Information Example

The imperfect-information example uses a positive or negative economic signal. The prior probabilities are $P(\mathrm{good}) = 0.6$ and $P(\mathrm{bad}) = 0.4$. The signal accuracy is expressed conditionally: $P(\mathrm{positive} \mid \mathrm{good}) = 0.8$, and $P(\mathrm{positive} \mid \mathrm{bad}) = 0.1$. Therefore $P(\mathrm{negative} \mid \mathrm{good})$ is 0.2 and $P(\mathrm{negative} \mid \mathrm{bad})$ is 0.9.

The marginal probability of a positive signal follows the law of total probability.

$$P(\mathrm{positive}) = 0.8(0.6) + 0.1(0.4) = 0.52.$$

$$P(\mathrm{negative}) = 1 - 0.52 = 0.48.$$

10.4 Updating Beliefs with Bayes' Rule

Bayes' rule updates the economic-state probabilities after each signal.

$$P(\text{good} \mid \text{positive}) = \frac{0.8(0.6)}{0.52} = \frac{12}{13} \approx 0.9231.$$

$$P(\text{bad} \mid \text{positive}) = \frac{1}{13} \approx 0.0769.$$

$$P(\text{good} \mid \text{negative}) = \frac{0.2(0.6)}{0.48} = 0.25.$$

$$P(\text{bad} \mid \text{negative}) = 0.75.$$

10.5 Optimal Decisions After Each Signal

After a positive signal, expected payoffs are approximately forty-eight point four six for apartment, eighty-one point two three for office, and twenty-eight point four six for warehouse. Office remains optimal. After a negative signal, expected payoffs are thirty-five for apartment, minus five for office, and fifteen for warehouse. Apartment becomes optimal.

$$V(\text{positive}) = \max\{48.46,\ 89.22,\ 28.46\} = 81.23.$$

$$V(\text{negative}) = \max\{35,\ -5,\ 15\} = 35.$$

10.6 Expected Value of Imperfect Information

Weighting these conditional optimal values by signal probabilities gives:

$$\text{EV}_{\text{imperfect}} = 0.52(89.22) + 0.48(35) = 63.1944.$$

$$\text{EVSI} = 63.1944 - 44 = 19.1944.$$

10.7 The Decision-Analysis Workflow

The complete workflow is: construct priors; specify signal likelihoods; calculate signal marginals; update posteriors with Bayes' rule; optimize separately after each signal; average the conditional optimal values; and subtract the uninformed benchmark. This combines probability modeling and optimization into a single decision-analysis procedure.

Value of information is a decision concept, not merely a prediction-accuracy concept. Information has value only when it can arrive before a decision and can sometimes change the chosen action. A forecast may be statistically informative yet have zero economic value if the same action remains optimal under every signal.

10.8 Comparing the Three Information Regimes

Three models should be compared on the same payoff scale. The no-information model chooses one action using prior state probabilities. The perfect-information model observes the true state and then chooses. The sample- or imperfect-information model observes a noisy signal, updates beliefs, and chooses a signal-specific action.

$$\text{EVPI} = \mathbb{E}_{\text{state}}\left[\max_a \ \text{payoff}(a, \text{state})\right] \ - \ \max_a \ \mathbb{E}[\text{payoff}(a, \text{state})].$$

The maximum is inside the expectation under perfect information because the action can depend on the realized state. Without information, the maximum is outside because a single action must be selected before the state is known. This noncommutativity is the source of information value.

For imperfect information Y, the value is:

$$\text{EVSI} = \sum_y P(y) \ \max_a \ \mathbb{E}[\text{payoff}(a, S) \mid Y = y] \ - \ \text{baseline EMV}.$$

The Bayes calculations and decision calculations must be kept separate. First calculate $P(\text{signal})$ using total probability. Second calculate posterior state probabilities for that signal. Third evaluate all actions under those posterior probabilities. Fourth select the best action. Only then average across signals.

In the stated example, a positive signal occurs with probability 0.52 and leads to office. A negative signal occurs with probability 0.48 and leads to apartment. The policy can be written: choose office after positive; choose apartment after negative. That policy is the operational output of the information analysis.

10.9 Bounds and Sensitivity

The imperfect-information value must satisfy a bound when the analysis is internally consistent.

$$0 \leq \text{EVSI} \leq \text{EVPI}.$$

The computed values, 19.1944 and 28, satisfy this ordering.

The value of perfect information is a ceiling on the price of any forecast, not an automatic purchase recommendation. The net value of a real information service is EVSI minus its acquisition and implementation costs. Delays, model maintenance, and the ability to act on the signal also matter.

Signal quality can be studied through sensitivity analysis. If $P(\text{positive} \mid \text{good})$ and $P(\text{negative} \mid \text{bad})$ both move toward one, posteriors become more extreme and EVSI approaches EVPI. If signal likelihoods become identical across states, the likelihood ratio approaches one, posteriors equal priors, the policy stops changing, and EVSI approaches zero.

Perfect information reverses node order by revealing the state before choice. Imperfect information inserts a signal node before choice but leaves the true state uncertain afterward. This distinction is critical: a noisy forecast does not replace the economic-state chance node; it changes the probabilities used at that later node.

The broader workflow supports market research, medical testing, quality inspection, forecasting, and machine-learning-assisted decisions. A model should report not only predictive performance but also the decisions it changes, the expected payoff improvement, and the maximum rational price of the information.

10.10 Rapid Technical Review

- A probability model assigns nonnegative probabilities that sum to one.
- Addition subtracts overlap; conditioning renormalizes within observed information.
- Bayes' rule reverses conditioning and updates priors to posteriors.
- Independence means conditioning does not change probability.
- Chance nodes take expected values; decision nodes optimize across actions.
- Rollback solves a tree from terminal nodes toward the root.
- EVPI compares perfect foresight with the uninformed optimum.
- Imperfect-information value requires Bayesian posteriors and signal-contingent decisions.

Part III

Random Variables, Statistical Distributions, and Statistical Estimation (Module 3)

11 | Random Variables, Probability Mass, Density, and Sampling

This chapter moves from events to random variables, sampling, and statistical inference.

11.1 From Events to Numerical Random Variables

An event is a subset of a sample space. A random variable goes one step further: it assigns a numerical value to every elementary outcome. Formally, if Ω is the sample space, a random variable X is a measurable mapping from Ω into the real numbers,

$$X : \Omega \to \mathbb{R}.$$

The mapping sends outcomes in the sample space into real numbers. The word *random* describes the unknown outcome before the experiment; the mapping itself is fixed. For a coin toss, one may define $X(\text{heads}) = 1$ and $X(\text{tails}) = 0$. The same experiment could support another valid encoding, but once an encoding is chosen it must be applied consistently.

A discrete random variable takes values in a finite or countably infinite set. Examples include the number of children in a family, a zero-one indicator, or the integer identifier assigned to a language token. A continuous random variable takes values across an interval or another uncountable set. Examples include time, temperature, a financial return, or an idealized pixel intensity.

$$\text{Discrete support: } S_X = \{x_1, x_2, \ldots\}, \qquad \text{Continuous support: } S_X \subset \mathbb{R}.$$

Generative artificial intelligence provides useful motivation. Text models convert text into tokens, associate tokens with discrete numerical identifiers, and generate a conditional distribution for the next token. Image, audio, and video systems often represent their data as high-dimensional numerical arrays. Although stored pixels are quantized on computers, mathematical models frequently treat latent variables or normalized intensities as continuous.

For an RGB image with height H and width W, a simple representation contains three values per pixel. A 256×256 image therefore contains $256 \times 256 \times 3 = 196{,}608$ scalar components. The joint random vector is enormously higher-dimensional than a single coin toss, which helps explain why training and inference are computationally expensive.

$$\text{Dimension of an } H \times W \text{ RGB image} = 3HW;$$
$$\text{for } H = W = 256, \text{ dimension} = 196{,}608.$$

11.2 Probability Mass Functions and Probability Density Functions

Every probability model must be nonnegative and normalized. For a discrete variable, the *probability mass function* (PMF) assigns a probability to each attainable value. The masses must sum to one.

$$p_X(x) = P(X = x), \qquad p_X(x) \geq 0, \qquad \sum_{x \in S_X} p_X(x) = 1.$$

For a continuous variable, the *probability density function* (PDF) is denoted f_X. A density is also nonnegative, but normalization is expressed by an integral rather than a sum.

$$f_X(x) \geq 0 \qquad \text{and} \qquad \int_{-\infty}^{\infty} f_X(x)\, dx = 1.$$

A central technical distinction is that density is not probability. A density may exceed one when its support is narrow. Probabilities are areas under the density curve and must lie between zero and one. For a genuinely continuous variable, the probability of any exact point is zero, even when the density at that point is positive.

$$P(a \leq X \leq b) = \int_a^b f_X(x)\, dx; \qquad \text{for continuous } X,\ P(X = x_0) = 0.$$

The word *likelihood* is sometimes used loosely for the height of a density, but density and likelihood should be distinguished. A density $f(x \mid \theta)$ is a function of possible data x when θ is fixed. A likelihood $L(\theta \mid x)$ treats the same mathematical expression as a function of the unknown parameter θ. Density describes curve height; likelihood is reserved for parameter inference.

$$\text{Density: } f(x \mid \theta) \text{ as a function of } x.$$
$$\text{Likelihood: } L(\theta \mid x) \propto f(x \mid \theta) \text{ as a function of } \theta.$$

11.3 Uniform Distributions and Probabilities as Areas

The first continuous example is the uniform distribution. If X is uniform on the interval from a to b, the density is constant inside that interval and zero outside it. The rectangle must have area one, so its height is the reciprocal of the interval length.

$$\text{If } X \sim \text{Uniform}(a, b), \text{ then } f_X(x) = \frac{1}{b-a} \text{ for } a \leq x \leq b, \text{ and 0 otherwise.}$$

For Uniform$(1, 10)$, the support has length nine, so the density is one ninth. A proposed constant height of five is positive but invalid because its area is five times nine, or forty-five, rather than one.

$$\int_1^{10} \frac{1}{9}\, dx = \frac{10-1}{9} = 1.$$

For the interval from seven to seven point one, the probability is the area of a thin rectangle: width 0.1 multiplied by height one ninth.

$$P(7 \leq X \leq 7.1) = \frac{7.1-7}{10-1} = \frac{0.1}{9} = \frac{1}{90} \approx 0.0111.$$

The narrow uniform distribution on the interval from one to one point one has width 0.1 and density ten. This density is greater than one but still valid because ten times 0.1 equals one. The probability between 1.05 and 1.06 is ten percent.

$$\text{If } X \sim \text{Uniform}(1, 1.1), \text{ then } P(1.05 \leq X \leq 1.06) = 10(0.01) = 0.10.$$

11.4 Sampling, Empirical Frequency, and Reproducibility

Repeated draws from a specified distribution can be generated in Excel or Python. A sample is a realized value from the random variable; a dataset of size n contains n such draws. If the draws are independent and identically distributed, the empirical proportion of observations falling in an event A estimates its model probability.

$$\widehat{P}_n(A) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\{X_i \in A\}.$$

In one set of one hundred simulated draws from Uniform$(1, 1.1)$, seven fell between 1.05 and 1.06, giving an empirical proportion of 0.07. This does not contradict the theoretical value 0.10. Sampling error is expected in a finite dataset. Under the law of large numbers, the empirical proportion converges to the true probability as n grows.

$$\widehat{P}_n(A) \to P(A) \text{ as } n \to \infty, \text{ under standard independent sampling assumptions.}$$

A pseudorandom seed initializes a deterministic number generator. Reusing the same seed with the same generator and software implementation reproduces the same sequence, which is valuable for validation and debugging. Cross-platform results may differ when implementations differ. A seed does not make observations more random; it makes a pseudorandom experiment reproducible.

This process is closely connected to AI inference. A generative model produces outputs by sampling from learned conditional distributions. The analogy should not be taken too literally: modern language models do not merely transform one scalar uniform distribution into English. They learn a high-dimensional parameterized network, compute conditional token probabilities with a softmax layer, and apply a decoding rule such as greedy selection, temperature sampling, top-k sampling, or nucleus sampling.

$$P(\text{next token} = j \mid \text{context}) = \frac{\exp(z_j/T)}{\sum_k \exp(z_k/T)}.$$

Here z_j is the model score for token j and T is temperature. Lower temperature concentrates probability on high-scoring tokens; higher temperature flattens the distribution and increases output diversity. This is the operational connection between probability distributions, sampling, and why the same prompt can produce different responses.

12 | Distribution Transformations, Quantiles, and Expected Value

12.1 Affine Transformations of Random Variables

A transformation constructs a new random variable from an existing one. If $Y = g(X)$, then each realized X value is converted by the deterministic function g. Shifting, squeezing, and stretching are the most common cases; the most important basic case is an affine transformation,

$$Y = aX + b.$$

The coefficient a changes scale. If $|a|$ exceeds one, the distribution is stretched; if it lies between zero and one, the distribution is compressed. A negative a also reverses orientation. The constant b shifts every value by the same amount. If X has support from lower bound L to upper bound U and $a > 0$, then Y has support from $aL + b$ to $aU + b$.

$$\text{If } a > 0 \text{ and } X \in [L, U], \text{ then } Y \in [aL + b,\ aU + b].$$

Consider mapping Uniform$(1, 10)$ to Uniform$(4, 6)$. One structured derivation uses three steps. First subtract one, moving the support from zero to nine. Second multiply by two ninths, compressing its length from nine to two. Third add four, shifting the support to four through six.

$$Y = 4 + \frac{2}{9}(X - 1) = \frac{2}{9}X + \frac{34}{9}.$$

Checking endpoints verifies the mapping. When $X = 1$, $Y = 4$. When $X = 10$, $Y = 6$. Because the input is uniform and the transformation is linear and increasing, the output is also uniform. Its density is one divided by two, or one half.

$$f_Y(y) = \frac{1}{2} \quad \text{for } 4 \le y \le 6.$$

For a differentiable one-to-one transformation $Y = g(X)$, density changes according to the Jacobian rule. This rule preserves total probability while horizontal distances are stretched or compressed.

$$f_Y(y) = f_X\left(g^{-1}(y)\right) \left| \frac{d}{dy} g^{-1}(y) \right|.$$

A valid random-variable transformation does not always have to be one-to-one. For example, $Y = X^2$ is valid. One-to-one behavior is required only for the simplest single-inverse change-of-variables formula. If several X values map to the same Y, their density contributions must be summed over all inverse branches.

$$f_Y(y) = \sum_{x:\, g(x)=y} \frac{f_X(x)}{|g'(x)|}, \qquad \text{when the inverse branches are regular.}$$

12.2 Uniform and Triangular Densities

Uniform and triangular shapes offer a useful comparison. A density is valid when it is nonnegative and its total area is one. For a triangle with base from one to ten, the base length is nine. If the peak height is h, normalization requires

$$\frac{1}{2}(9)h = 1, \qquad \text{so } h = \frac{2}{9}.$$

Moving the triangular peak changes where probability mass is concentrated but not the height needed for normalization, as long as the base endpoints remain one and ten and the triangle still returns to zero at both endpoints. A peak near the left creates a right-skewed distribution with a long right tail. A peak near the right creates a left-skewed distribution. A peak at the midpoint 5.5 produces a symmetric triangular distribution.

For the symmetric triangular case, the mean and median both equal 5.5. For a skewed triangular distribution they generally differ. The mode is the location of the density peak, the median divides probability mass in half, and the mean is the balance point. These three measures answer different questions and need not coincide.

$$\text{Symmetric distribution about } c \implies \text{mean} = \text{median} = c \text{ when the mean exists;}$$
$$\text{for the symmetric triangle, } c = 5.5.$$

12.3 Median, Cumulative Probability, and the Discrete Convention

The median is a quantile. A continuous median m satisfies equal probability on either side when the density is continuous and has no flat ambiguity. More generally, a median satisfies that at least half the mass is at or below m and at least half is at or above m.

$$P(X \le m) \ge 0.5 \qquad \text{and} \qquad P(X \ge m) \ge 0.5.$$

Using the cumulative distribution function $F_X(x) = P(X \le x)$, the course convention for a discrete median is the smallest attainable value whose cumulative probability reaches or exceeds one half.

$$m = \inf\{x : F_X(x) \ge 0.5\}.$$

The family-size distribution assigns probabilities 0.15, 0.20, 0.35, and 0.30 to zero, one, two, and three children. The cumulative probabilities are 0.15, 0.35, 0.70, and 1. The first cumulative value reaching one half occurs at two, so the course median is two children.

$$F(1) = 0.35 < 0.5, \qquad \text{while} \qquad F(2) = 0.70 \ge 0.5; \qquad \text{therefore median} = 2.$$

12.4 Expected Value as a Probability-Weighted Balance Point

Expected value, also called the mean and written μ, is the probability-weighted average of possible outcomes. For a discrete variable it is a sum. For a continuous variable it is an integral against the density.

$$E[X] = \sum_x x\, p_X(x), \qquad \text{for discrete } X,$$

$$E[X] = \int_{-\infty}^{\infty} x\, f_X(x)\, dx, \qquad \text{for continuous } X.$$

The center-of-mass interpretation is useful. Imagine each possible value placed on a balance beam with a mass equal to its probability. The expected value is the balance point. Unlike the discrete median, the expected value is unique whenever it exists, but it need not be an attainable outcome. A family cannot have 1.8 children, yet 1.8 is a meaningful population average.

For the stated family distribution, the weighted mean is

$$E[X] = 0(0.15) + 1(0.20) + 2(0.35) + 3(0.30) = 1.80.$$

Expectation is linear even when variables are dependent. Constants move through the expectation operator in the natural way.

$$E[aX + b] = aE[X] + b, \qquad E[X_1 + \cdots + X_n] = E[X_1] + \cdots + E[X_n].$$

For one thousand families with the same expected family size, the expected total number of children is $1000 \times 1.8 = 1800$. If each child is assumed independently to be a girl with probability one half, the expected number of girls is 900. Linearity supplies the expected total; independence is not required for the addition of expectations, although the one-half conditional assumption is needed for the gender calculation.

$$E[\text{total children}] = 1000(1.8) = 1800; \qquad E[\text{girls}] = 0.5(1800) = 900.$$

This result illustrates a recurring theme: transformations of random variables induce transformations of summary measures. The mean responds linearly to shifts and changes of scale, while the next chapter shows that variance responds quadratically to scale.

13 | Variance, Standard Deviation, Dependence, and Risk

13.1 Why the Mean Is Not Enough

Two distributions can have the same expected value but very different risk. Consider lottery W, which pays -10 or $+10$ with equal probability, and lottery Y, which pays -100 or $+100$ with equal probability. Both means are zero, but Y has much greater dispersion and therefore exposes the decision maker to larger gains and losses.

$$E[W] = (-10)(0.5) + (10)(0.5) = 0, \qquad E[Y] = (-100)(0.5) + (100)(0.5) = 0.$$

Preference between these lotteries cannot be explained by expected value alone. A risk-averse person may prefer W, while a risk-seeking person may prefer Y. Variance and standard deviation describe dispersion, but they do not by themselves specify preferences. A complete decision model could apply expected utility, downside risk, value at risk, or another criterion.

$$\text{Expected utility} = \sum_x p(x)u(x), \qquad \text{where concave } u \text{ represents risk aversion.}$$

13.2 Variance and Standard Deviation

Variance is the expected squared deviation from the mean. If $\mu = E[X]$, subtracting μ centers each realization, squaring removes signs and emphasizes large deviations, and probability weighting recognizes that frequent deviations matter more than rare ones.

$$\text{Var}(X) = \sigma_X^2 = E\Big[(X - \mu_X)^2\Big].$$

For a discrete random variable, variance is calculated as a probability-weighted sum. For a continuous variable, it is an integral.

$$\text{Var}(X) = \sum_x (x - \mu_X)^2 \, p_X(x), \qquad \text{Var}(X) = \int_{-\infty}^{\infty} (x - \mu_X)^2 \, f_X(x)\, dx.$$

Variance has squared units. If X is measured in dollars, variance is measured in dollars squared. Standard deviation returns to the original units and is therefore easier to interpret.

$$SD(X) = \sigma_X = \sqrt{\text{Var}(X)}.$$

For W, both outcomes are ten units from the zero mean. The variance is one hundred and the standard deviation is ten.

$$\text{Var}(W) = (-10 - 0)^2(0.5) + (10 - 0)^2(0.5) = 100, \qquad SD(W) = \sqrt{100} = 10.$$

For equally likely Y values -100 and $+100$, the variance is ten thousand and the standard deviation is one hundred. This is consistent with $Y = 10W$.

$$\text{Var}(Y) = 10{,}000 \quad \text{and} \quad SD(Y) = 100.$$

13.3 The Computational Identity and Transformation Rules

Expanding the square gives an equivalent variance formula. It is often computationally convenient, although the centered formula is usually more numerically stable in software when the mean is very large relative to the spread.

$$\operatorname{Var}(X) = E[X^2] - (E[X])^2.$$

Shifting every outcome by a constant changes the mean but not the variance. Scaling every outcome by a multiplies variance by a^2 and standard deviation by $|a|$.

$$E[aX + b] = aE[X] + b, \qquad \operatorname{Var}(aX + b) = a^2 \operatorname{Var}(X), \qquad SD(aX + b) = |a|\, SD(X).$$

The absolute value matters for standard deviation because standard deviation cannot be negative. If $a = -2$, the distribution is reflected and stretched by two; its standard deviation doubles, and does not become negative.

Suppose the Y lottery is modified so that -100 occurs with probability 0.4 and $+100$ occurs with probability 0.6. The mean shifts to twenty because the positive outcome is more likely.

$$E[Y] = (-100)(0.4) + (100)(0.6) = 20.$$

The corresponding variance is nine thousand six hundred, and the standard deviation is approximately 97.98.

$$\operatorname{Var}(Y) = (-100 - 20)^2(0.4) + (100 - 20)^2(0.6) = 9{,}600, \qquad SD(Y) = \sqrt{9{,}600} \approx 97.98.$$

The decrease from variance ten thousand to nine thousand six hundred reflects the shifted mean and unequal masses. The $+100$ outcome is now closer to the mean and also more likely, reducing average squared deviation modestly.

13.4 Adding Random Variables: Independence and Covariance

Variance is not linear in the same way as expectation. For independent random variables, the variance of a weighted sum is the sum of weighted variances, with each coefficient squared.

$$\text{If } X_1 \text{ and } X_2 \text{ are independent,} \qquad \operatorname{Var}(c_1X_1 + c_2X_2) = c_1^2 \operatorname{Var}(X_1) + c_2^2 \operatorname{Var}(X_2).$$

For dependent variables, covariance captures how their deviations move together. Positive covariance means above-average values tend to occur together; negative covariance means one tends to be above average when the other is below average.

$$\operatorname{Cov}(X_1, X_2) = E[(X_1 - \mu_1)(X_2 - \mu_2)]\,,$$

$$\operatorname{Var}(c_1X_1 + c_2X_2) = c_1^2 \operatorname{Var}(X_1) + c_2^2 \operatorname{Var}(X_2) + 2c_1c_2 \operatorname{Cov}(X_1, X_2).$$

Independence implies zero covariance when second moments exist, but zero covariance does not generally imply independence. This distinction matters in regression, portfolio analysis, forecasting, and any system where multiple uncertain inputs move together.

The failure of the independent-sum formula is illustrated with $X_2 = 3X_1 + 5$. Because X_2 is constructed directly from X_1, the variables are perfectly dependent. First, the variance of X_2 is nine times the variance of X_1.

$$X_2 = 3X_1 + 5 \implies \operatorname{Var}(X_2) = 9 \operatorname{Var}(X_1).$$

Adding them gives $4X_1 + 5$. Therefore the exact variance is sixteen times the variance of X_1, not ten times.

$$\operatorname{Var}(X_1 + X_2) = \operatorname{Var}(4X_1 + 5) = 16 \operatorname{Var}(X_1).$$

The missing six units of variance come from twice the covariance. Since $\operatorname{Cov}(X_1, 3X_1+5) = 3 \operatorname{Var}(X_1)$, the covariance term is six variance units.

$$2 \operatorname{Cov}(X_1, X_2) = 2 \times 3 \operatorname{Var}(X_1) = 6 \operatorname{Var}(X_1).$$

Covariance depends on measurement units, so correlation is often used to express standardized dependence. Correlation divides covariance by the two standard deviations and therefore lies between -1 and $+1$. Values near $+1$ indicate strong positive linear co-movement, values near -1 indicate strong negative linear co-movement, and values near zero indicate little linear association.

$$\operatorname{Corr}(X_1, X_2) = \frac{\operatorname{Cov}(X_1, X_2)}{SD(X_1)\, SD(X_2)}, \qquad -1 \leq \operatorname{Corr} \leq 1.$$

For $X_2 = 3X_1 + 5$, assuming X_1 has positive variance, the correlation is $+1$ because X_2 is a perfectly increasing affine function of X_1. If the coefficient had been negative, the correlation would be -1. This makes the dependence failure in the variance calculation visible on a standardized scale.

The risk calculation also distinguishes population moments from estimates. The formulas above describe the true distributional mean and variance. With observed data, analysts estimate them using a sample mean and sample variance. The common unbiased sample variance divides the sum of squared deviations by $n - 1$ because the sample mean was estimated from the same data.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i; \qquad s^2 = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2.$$

This is the key modeling lesson: whether risks diversify depends on dependence. Adding independent risks can smooth relative uncertainty, while adding highly positively correlated risks does not provide the same diversification benefit.

14 | The Normal Distribution, Standardization, and Z Probabilities

14.1 The Normal Density and Its Normalization

Unlike the uniform examples with finite lower and upper bounds, a normal random variable has support across the entire real line. Its density is symmetric, peaks at the mean μ, and decays rapidly as x moves away from μ. The parameter σ is the standard deviation, and σ^2 is the variance.

$$X \sim N(\mu, \sigma^2).$$

The normal probability density is

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right], \qquad -\infty < x < \infty.$$

The exponential term creates the bell shape. Squaring $x - \mu$ makes the curve symmetric around μ. The negative sign causes density to decrease as squared distance grows. The factor $1/(\sigma\sqrt{2\pi})$ is the normalizing constant that makes the total area equal one.

$$\int_{-\infty}^{\infty} f_X(x)\, dx = 1.$$

Changing μ shifts the entire curve without changing its shape. Increasing σ spreads the same total probability over a wider range, lowering the peak. Decreasing σ concentrates probability around the mean and raises the peak. For every normal distribution, symmetry implies that mean, median, and mode coincide at μ.

$$\text{For a normal distribution,} \qquad \text{mean} = \text{median} = \text{mode} = \mu.$$

14.2 The Empirical Rule and Tail Behavior

Normal distributions are unbounded, but most probability mass lies within a few standard deviations of the mean. The standard empirical rule gives approximately 68.27% within one standard deviation, 95.45% within two, and 99.73% within three.

$$P(\mu - \sigma \le X \le \mu + \sigma) \approx 0.6827,$$

$$P(\mu - 2\sigma \le X \le \mu + 2\sigma) \approx 0.9545,$$

$$P(\mu - 3\sigma \le X \le \mu + 3\sigma) \approx 0.9973.$$

These are the canonical 68–95–99.7 normal-distribution percentages, and this book uses these standard values throughout.

The probability outside plus or minus three standard deviations is approximately 0.0027 in total, or about 0.00135 in each tail. Rare does not mean impossible: the normal model still assigns positive probability to arbitrarily extreme finite observations.

$$P(|X - \mu| > 3\sigma) \approx 1 - 0.9973 = 0.0027.$$

14.3 Affine Transformations of Normal Variables

Normal distributions are closed under affine transformations. If X is normal and $Y = aX + b$, then Y is also normal. Its mean is transformed linearly and its variance is multiplied by a^2.

$$\text{If } X \sim N(\mu, \sigma^2), \text{ then } aX + b \sim N(a\mu + b,\ a^2\sigma^2).$$

The corresponding standard deviation is the absolute value of a times σ. A pure shift $X + b$ changes only the mean. A pure scale aX changes both the mean and the spread.

$$E[aX + b] = a\mu + b; \qquad SD(aX + b) = |a|\,\sigma.$$

Standardization converts any nondegenerate normal variable to the standard normal distribution. Subtracting μ shifts the mean to zero, and dividing by σ rescales the standard deviation to one.

$$Z = \frac{X - \mu}{\sigma}, \qquad Z \sim N(0, 1).$$

The standard normal cumulative distribution function, written Φ, returns the left-tail probability. A Z table is a printed lookup table for Φ.

$$\Phi(z) = P(Z \le z) = \int_{-\infty}^{z} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{t^2}{2}\right) dt.$$

Symmetry connects positive and negative table values. The left-tail probability at $-z$ equals one minus the left-tail probability at $+z$.

$$\Phi(-z) = 1 - \Phi(z).$$

14.4 Worked Z-Table Calculations

Suppose X is normal with mean one hundred and standard deviation fifty. To find the probability that X is at most one hundred ten, standardize the cutoff.

$$z = \frac{110 - 100}{50} = 0.20.$$

The Z table gives $\Phi(0.20) \approx 0.57926$. Thus about 57.9% of this distribution lies at or below 110.

$$P(X \le 110) = \Phi(0.20) = 0.57926.$$

For an interval probability, standardize both endpoints and subtract cumulative left-tail areas. The interval from ninety to one hundred ten maps to -0.20 through $+0.20$.

$$P(90 \le X \le 110) = P(-0.20 \le Z \le 0.20) = \Phi(0.20) - \Phi(-0.20).$$

Using the table values 0.57926 and 0.42074, the desired central area is

$$0.57926 - 0.42074 = 0.15852.$$

The same method handles any interval. For a right-tail probability, subtract a left-tail table value from one. For a two-sided probability, standardize both bounds and subtract. It is important to always state whether the second normal parameter denotes variance or standard deviation; this book writes $N(\mu, \sigma^2)$ so that the standard deviation used in a Z score is σ.

$$P(X > c) = 1 - \Phi\left(\frac{c-\mu}{\sigma}\right), \qquad P(a \le X \le b) = \Phi\left(\frac{b-\mu}{\sigma}\right) - \Phi\left(\frac{a-\mu}{\sigma}\right).$$

These normal calculations prepare for confidence intervals and statistical inference, taken up in the chapters that follow. Standardization converts a problem with arbitrary units into a universal reference scale. Once the distribution of a standardized statistic is known, table values or software can translate distances from the mean into probabilities, tail areas, critical values, and confidence levels.

14.5 Chapter Summary

The chapter can be summarized as a pipeline: first define a numerical random variable and its support; second, specify a normalized PMF or PDF; third, distinguish theoretical probabilities from finite-sample frequencies; fourth, use transformations to connect variables and derive their means and variances; fifth, when a normal model is appropriate, standardize to Z and calculate areas with the standard normal distribution.

Model → distribution → sample → summary measures → standardized inference.

The AI examples fit the first half of this pipeline: model outputs are random variables, softmax supplies normalized conditional masses, and decoding samples from those masses. The statistical examples fit the second half: expectation summarizes center, variance summarizes spread, covariance captures dependence, and standardization makes normal probabilities comparable across measurement scales.

Rapid Review

- A random variable maps experimental outcomes to numbers; it may be discrete or continuous.
- A PMF sums to one. A PDF integrates to one, and probability is area under the density.
- For Uniform(a, b), density is $1/(b-a)$.
- An affine transformation $Y = aX + b$ shifts and rescales a distribution.
- The mean is a probability-weighted balance point; expectation is linear.
- Variance is expected squared distance from the mean; standard deviation is its square root.
- Scaling by a multiplies variance by a^2; shifting by b does not change variance.

- Variances add directly only under zero covariance; independence is a sufficient condition.
- A normal variable has bell-shaped density parameterized by μ and σ^2.
- Standardization $Z = (X - \mu)/\sigma$ converts normal probabilities to the $N(0, 1)$ reference scale.

Core Formula Sheet

- Uniform density: $f_X(x) = 1/(b - a)$, for $a \le x \le b$.
- Expectation: $E[X] = \sum x\, p_X(x)$, or $\int x f_X(x)\, dx$.
- Variance: $\mathrm{Var}(X) = E[(X - \mu)^2] = E[X^2] - \mu^2$.
- Affine transformation: $E[aX + b] = aE[X] + b$; $\mathrm{Var}(aX + b) = a^2 \mathrm{Var}(X)$.
- Covariance sum rule: $\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X, Y)$.
- Normal density: $f(x) = \left[1/(\sigma\sqrt{2\pi})\right] \exp\left[-(x - \mu)^2/(2\sigma^2)\right]$.
- Standardization: $Z = (X - \mu)/\sigma$.
- Interval probability: $P(a \le X \le b) = \Phi((b - \mu)/\sigma) - \Phi((a - \mu)/\sigma)$.

15 | From a Population Distribution to Statistical Estimation

The easiest way to understand this chapter is to see it as answering one fundamental question: *what can we learn about an unknown population when we cannot observe the entire population?* That question takes us from probability theory into statistics.

15.1 From a Known Distribution to an Unknown One

In the previous chapters, the starting point was a probability distribution. If we know the probability distribution of a random variable X, then we can calculate quantities such as its expected value, variance, and standard deviation.

For a continuous random variable, a probability density must satisfy two fundamental requirements: it must be nonnegative, and the total area underneath the density must equal one. An important distinction is that the density itself does not necessarily have to be below one — what must lie between zero and one is probability, obtained by calculating areas under the density.

Once the distribution is known, the expected value is conceptually straightforward: a probability-weighted average of possible outcomes. For a discrete random variable,

$$E[X] = \sum_x xP(x).$$

So expected value is simply a weighted average, where the probabilities provide the weights.

Variance uses the same basic idea but asks a different question. Instead of asking where the distribution is centered, variance asks how dispersed the outcomes are around that center. Thus,

$$\text{Var}(X) = E[(X - \mu)^2].$$

We first calculate the difference between an observation and the population mean, square that difference, and then take its probability-weighted average. Squaring is important because deviations can be positive or negative; we care about their magnitude rather than their sign, and squaring also avoids some of the mathematical complications associated with absolute values.

Variance is therefore always nonnegative. It can equal zero only in the degenerate case where every observation is exactly the same. As variation among observations increases, variance increases.

The problem with variance is its unit. If X is measured in dollars, variance is measured in dollars squared. Therefore, we take its square root and obtain the standard deviation,

$$\sigma = \sqrt{\text{Var}(X)}.$$

Standard deviation brings uncertainty back into the original units of the variable.

15.2 The Fundamental Practical Problem

All of these formulas appear simple when the population probability distribution $P(x)$ is known. But in real applications, **we usually do not know** $P(x)$. That is the central problem motivating the rest of this chapter.

Building or observing the complete probability distribution can be extraordinarily expensive or practically impossible. A population need not consist of people: it might consist of possible sales outcomes, words or sentences in a language, images, driving environments, customer behavior, or almost anything else that we want to model.

So instead of observing the entire population distribution, we collect a random sample:

$$X_1, X_2, \ldots, X_n.$$

This leads to one of the most important tricks in analytics. Rather than explicitly estimating the entire probability distribution $P(x)$, we replace probability weighting by equal weighting of randomly sampled observations. Each sampled observation receives weight

$$\frac{1}{n}.$$

Consequently, $E[X]$ is estimated by

$$\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i.$$

This is the sample mean. The deeper message is more important than this particular formula: whenever an unknown population quantity can be represented as an expectation under $P(x)$, random sampling gives us a way of approximating that expectation without explicitly knowing the entire distribution. This is why ordinary sample averages connect to **Monte Carlo estimation** — sample means and sample variances can be viewed as simple applications of the much broader Monte Carlo principle.

15.3 Why Random Sampling Is Doing the Work

There is a crucial qualification: the observations must actually be collected in an appropriately random fashion. Suppose the sample consists of 100 observations. Giving each observation weight 1/100 creates an empirical distribution over those observations. The hope is that this empirical distribution becomes increasingly representative of the underlying population as the sample becomes larger.

Consider an MBA salary dataset. The population contains more than 2,000 observations, and the population mean salary is approximately 111,208. A random sample of 100

observations produces a sample mean of approximately 111,568. Those numbers are not identical, and they should not be identical: sampling introduces randomness. If we drew another sample of 100 people, we would obtain another sample mean, and another sample would give yet another.

But something very important happens as the sample size increases: the sample mean tends to get closer to the population mean. This leads naturally to the **Law of Large Numbers**. In simplified notation,

$$\bar{X} \to \mu \qquad \text{as} \qquad n \to \infty.$$

The sample does not need to reproduce the population perfectly at any particular finite sample size. Instead, the procedure has the desirable property that increasing the amount of data moves us toward the population quantity.

15.4 The Important Complication: Estimating Variance

If the population mean μ were known, we could estimate variance by averaging $(X_i - \mu)^2$. But this creates an obvious problem: if we knew the true population mean, we would already know something about the population that we are trying to estimate. In practice, μ is unknown, so we replace it with the sample mean $\bar{X}$, giving squared deviations $(X_i - \bar{X})^2$.

But using $\bar{X}$ creates another consequence: these deviations are no longer completely free. Once the sample mean is calculated, the deviations around that sample mean must satisfy

$$\sum_{i=1}^{n}(X_i - \bar{X}) = 0.$$

If you know $n - 1$ of those deviations, the final deviation is automatically determined because all deviations must add to zero. Therefore, although there are n observations, there are only $n - 1$ independent deviations. We say that we have lost **one degree of freedom**. That is the intuition behind the familiar sample variance formula,

$$S^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2.$$

Why not divide by n? Because $\bar{X}$ was itself estimated using the same data. Dividing by n would systematically make the estimated variance too small. The $n - 1$ adjustment compensates for this tendency. This idea becomes even more important later in regression, where the same dataset is used to estimate multiple parameters and consequently additional degrees of freedom are consumed. The general lesson: every time the data are used to estimate an unknown quantity that subsequently enters another calculation, degrees of freedom must be tracked carefully.

15.5 Accuracy Improves Surprisingly Slowly

The next major idea concerns how quickly statistical estimates improve as more data are collected. Suppose 100 observations give us some estimation error. If we want to cut that

error in half, doubling the sample size is *not* sufficient — we need approximately four times as much data. This is the **square-root relationship**: statistical error generally shrinks according to

$$\frac{1}{\sqrt{n}}.$$

Therefore, if $n \to 4n$, then

$$\frac{1}{\sqrt{n}} \to \frac{1}{2\sqrt{n}}.$$

So quadrupling the sample size approximately halves the uncertainty. Likewise, if we want the error to become one-fourth as large, we need approximately sixteen times the sample size. More data help, but there are diminishing returns to additional data: moving from a poor estimate to a reasonably good estimate may be relatively inexpensive, while moving from a very good estimate to an extremely precise estimate can require enormous amounts of additional data. The same intuition applies to modern machine learning and AI, where reducing estimation error further and further can require very large increases in the amount of training data.

15.6 The Assumption Underneath Everything: Randomness and Independence

There is one condition underneath almost everything discussed so far: the sample must behave like a **random sample from the population**. If the data collection mechanism systematically favors certain observations, the empirical distribution created by assigning $1/n$ weight to observations may no longer properly represent the population.

Postal codes would generally be a poor mechanism for randomly sampling incomes because geography and income can be related. Similarly, conducting an election survey only in neighborhoods with particular political characteristics would obviously distort the results. The broader point is not about postal codes or elections; it is about the data-generating mechanism. Before applying statistical formulas, we should ask: how did these observations enter the dataset? Were observations selected independently? Was some part of the population systematically more likely to appear? Does observing one individual affect which individual will be observed next? Are there clusters or other dependence structures?

If the random-sampling assumption fails, simply increasing the sample size does not necessarily fix the problem. A huge biased sample can still produce an extremely precise estimate of the *wrong* quantity. Randomization, reshuffling, and resampling are practical ways of trying to preserve the statistical logic underlying estimation. In real company datasets, this becomes especially important because database records are rarely generated specifically for statistical inference.

15.7 The Conceptual Transition

We can now summarize the first major movement of this chapter. We began with the population, $P(X)$. If $P(X)$ were known, we could calculate μ, σ^2, σ directly. But $P(X)$ is usually unknown, so we take a random sample, $X_1, \ldots, X_n$. From that sample we

calculate statistics such as $\bar{X}$ and S^2. The sample mean estimates the population mean. The sample variance estimates the population variance, with the $n-1$ correction because the sample mean has itself been estimated from the same data. As n increases, these estimates become increasingly informative about their population counterparts.

But now a new question arises. Suppose our sample tells us that the mean is 111,568. We know that this number is almost certainly not exactly equal to the true population mean. So how uncertain should we be about it? In other words, **how do we quantify the error around a point estimate?** Instead of reporting only $\bar{X}$, we want something like

$$\bar{X} \pm \text{margin of error.}$$

To determine that margin of error, we need to understand the probability distribution of the estimator $\bar{X}$ itself. That brings us to one of the central results of statistics: the **Central Limit Theorem**, the subject of the next chapter.

16 | Sampling Distributions and the Central Limit Theorem

The previous chapter ended with a new problem. We know how to estimate the population mean using the sample mean,

$$\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i.$$

But once we calculate a sample mean, say 111,568, we know that this number is only an estimate. A different random sample would generally produce a different sample mean. So the key question becomes: **how much does the sample mean itself vary from sample to sample?** That question leads us to the concept of a **sampling distribution**.

16.1 The Sample Mean Is Itself a Random Variable

Suppose that we repeatedly draw random samples of size n from the same population. From the first sample, we calculate one sample mean; from the second, another; from the third, another. So we would obtain

$$\bar{X}_1, \bar{X}_2, \bar{X}_3, \ldots$$

These sample means vary because the underlying samples vary. Therefore, $\bar{X}$ is itself a random variable, and like every random variable, it has its own probability distribution. That distribution is called the **sampling distribution of the sample mean**.

This is conceptually different from the original population distribution. The population distribution describes the individual observations X. The sampling distribution describes the possible values of the statistic $\bar{X}$ that we would obtain across repeated samples. This distinction is one of the most important ideas in statistical inference.

16.2 Expected Value and Variance of the Sample Mean

What is $E[\bar{X}]$? Recall that $\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$. Taking expectations,

$$E[\bar{X}] = E\left[\frac{1}{n}\sum_{i=1}^{n} X_i\right].$$

Because expectation is linear,

$$E[\bar{X}] = \frac{1}{n}\sum_{i=1}^{n} E[X_i].$$

Each X_i comes from the same population, so $E[X_i] = \mu$. Therefore,

$$E[\bar{X}] = \frac{1}{n}\sum_{i=1}^{n}\mu = \mu,$$

since there are n copies of μ. This is the mathematical reason that the sample mean is centered at the true population mean, and the key reason why $\bar{X}$ is a good estimator of μ: if we repeatedly took samples and averaged their sample means, that long-run average would equal the population mean.

The next question is even more important for confidence intervals: how variable is $\bar{X}$? We start again with $\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$, so

$$\mathrm{Var}(\bar{X}) = \mathrm{Var}\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right).$$

Assuming the observations are independent,

$$\mathrm{Var}\left(\sum_{i=1}^{n} X_i\right) = \sum_{i=1}^{n}\mathrm{Var}(X_i).$$

The $1/n$ outside the sum enters the variance squared, so

$$\mathrm{Var}(\bar{X}) = \frac{1}{n^2}\sum_{i=1}^{n}\mathrm{Var}(X_i).$$

Each observation has population variance σ^2, therefore

$$\mathrm{Var}(\bar{X}) = \frac{1}{n^2}n\sigma^2 = \frac{\sigma^2}{n}.$$

This result is fundamental. Taking the square root, the standard deviation of $\bar{X}$ is

$$\sqrt{\frac{\sigma^2}{n}} = \frac{\sigma}{\sqrt{n}}.$$

This quantity is the standard deviation of the sampling distribution of the sample mean. In practical statistics, we usually call it the **standard error of the mean**,

$$SE(\bar{X}) = \frac{\sigma}{\sqrt{n}}.$$

The sample-mean distribution is much narrower than the original population distribution because averaging reduces noise: an individual observation can vary substantially, but an average of many observations is more stable. This is exactly why sample means become more precise as sample size increases.

Notice that the standard error, $\sigma/\sqrt{n}$, follows the same square-root relationship discussed in the previous chapter. If we multiply the sample size by four, $n \to 4n$, then

$$SE = \frac{\sigma}{\sqrt{4n}} = \frac{1}{2}\frac{\sigma}{\sqrt{n}},$$

so the standard error is cut in half. If we multiply the sample size by sixteen, SE is divided by four. This is the mathematical reason behind the earlier statement that halving estimation error requires approximately quadrupling the sample size.

16.3 The Central Limit Theorem

We already know two things about the sampling distribution of $\bar{X}$: $E[\bar{X}] = \mu$ and $SD(\bar{X}) = \sigma/\sqrt{n}$. But we still need to know its **shape**. The Central Limit Theorem gives us the answer: as the sample size becomes sufficiently large, the distribution of $\bar{X}$ approaches a normal distribution,

$$\bar{X} \approx N\left(\mu, \frac{\sigma}{\sqrt{n}}\right).$$

The crucial point is that this happens even if the underlying population itself is not normally distributed. The population may be uniform, right-skewed, left-skewed, or have some other unusual shape. Nevertheless, as the sample size increases, the distribution of the sample mean increasingly resembles a bell-shaped normal distribution. That is the extraordinary power of the Central Limit Theorem.

This theorem gives us a standardized way to quantify uncertainty. We know that approximately 95 percent of the standard normal distribution's probability lies between -1.96 and $+1.96$. So after standardizing the sample mean,

$$Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}},$$

we know that approximately

$$P\left(-1.96 \leq \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \leq 1.96\right) = 0.95.$$

This is the bridge from the Central Limit Theorem to confidence intervals.

16.4 Turning the Probability Statement Around

We do not actually want an interval for $\bar{X}$: we observe $\bar{X}$. What is unknown is μ. So we rearrange the inequality algebraically so that μ appears in the middle. This produces

$$P\left(\bar{X} - 1.96\frac{\sigma}{\sqrt{n}} \leq \mu \leq \bar{X} + 1.96\frac{\sigma}{\sqrt{n}}\right) = 0.95.$$

This is the standard 95 percent confidence interval for a population mean when the population standard deviation is known. We therefore define

$$\text{Lower Bound} = \bar{X} - 1.96\frac{\sigma}{\sqrt{n}}, \qquad \text{Upper Bound} = \bar{X} + 1.96\frac{\sigma}{\sqrt{n}}.$$

The quantity $1.96\,\sigma/\sqrt{n}$ is called the **margin of error**, so the entire expression can be remembered as

$$\boxed{\text{Confidence interval} = \text{Point estimate} \pm \text{Margin of error}} \quad \text{or} \quad \boxed{\bar{X} \pm 1.96\frac{\sigma}{\sqrt{n}}}.$$

16.5 Point Estimator versus Interval Estimator

The sample mean itself, $\bar{X}$, is a **point estimator**: it gives one number. But one number alone tells us nothing about how precise that estimate is. The confidence interval is an **interval estimator**: it gives a range around the point estimate.

This distinction is very important. Two studies might both estimate the same mean, say $\bar{X} = 100$ in Study A and $\bar{X} = 100$ in Study B, yet have very different confidence intervals: $[99, 101]$ for Study A versus $[70, 130]$ for Study B. The point estimates are identical, but Study A provides far more precise information. That is why reporting uncertainty is so valuable.

16.6 What Does 95 Percent Confidence Mean?

Confidence level is the probability with which the interval procedure captures the unknown population parameter. For a 95 percent confidence interval, the central area of the relevant sampling distribution is 95 percent, and the normal distribution leaves 5% outside the interval. Because the standard confidence interval is symmetric, this 5 percent is divided equally: 2.5% in the left tail and 2.5% in the right tail. This means the cumulative probability up to the positive cutoff is 0.975. Looking this up in the Z-table gives $Z = 1.96$; likewise, the lower cutoff is -1.96.

The margin of error is $MOE = Z\,\sigma/\sqrt{n}$. Suppose the confidence level and population variability stay fixed. As n increases, $\sqrt{n}$ increases, so MOE decreases and the confidence interval becomes narrower — more data produce more precise estimates, although the improvement is only at the square-root rate.

There is another tradeoff. Suppose we move from a 95 percent confidence level to 99 percent. To capture more of the sampling distribution, we must move further into the tails, which requires a larger critical value. A larger critical value produces a larger margin of error, so **higher confidence produces a wider confidence interval**. There is always a tradeoff between confidence and precision: to be more confident that the procedure covers the true parameter, we must accept a wider interval unless we collect more data.

16.7 The Salary Example

Applying this procedure to the MBA salary data: a sample of size $n = 100$ produced a sample mean of roughly 111,568. The population standard deviation, available here for teaching purposes because the entire population dataset is available, is approximately 47,506. For 95 percent confidence, $Z = 1.96$. The margin of error is

$$1.96 \times \frac{47{,}506}{\sqrt{100}}.$$

Since $\sqrt{100} = 10$, the standard error is approximately 4,751, and the margin of error is roughly 9,312. So the confidence interval is approximately

$$111{,}568 \pm 9{,}312.$$

The important conceptual structure, more than the precise arithmetic, is

$$\boxed{\bar{X} \pm Z\frac{\sigma}{\sqrt{n}}}.$$

16.8 But There Is a Major Problem

The formula above contains σ, the **population standard deviation**. If we knew the entire population well enough to calculate σ, we would often have far more information than we normally have in real statistical problems. Usually, the population standard deviation is unknown, so we estimate it using the sample standard deviation, S. But now we are estimating two population quantities from the sample: the mean, μ, and the standard deviation, σ. That extra estimation introduces additional uncertainty. Therefore, we should not continue using the normal Z-distribution exactly as before; we need a distribution with heavier tails to acknowledge that extra uncertainty. That distribution is the **Student t-distribution**, the subject of the next chapter.

17 | Confidence Intervals, Z versus t, and Sample-Size Calculation

We ended the previous chapter with an important practical problem. The confidence interval for a population mean was derived as

$$\bar{X} \pm Z\frac{\sigma}{\sqrt{n}}.$$

This formula works when the population standard deviation σ is known. But in real applications, that is often unrealistic: usually we do not know the population mean, and we also do not know the population standard deviation. What changes when σ is unknown? The answer is that we estimate σ using the sample standard deviation, and because this introduces additional uncertainty, we replace the normal Z-distribution with the **Student t-distribution**.

17.1 Estimating the Unknown Population Standard Deviation

Recall the sample standard deviation,

$$S = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2}.$$

The $n-1$ appears because $\bar{X}$ has already been estimated from the same data. Once σ is unknown, we substitute S for it, but we cannot simply write $\bar{X} \pm Z\,S/\sqrt{n}$ and continue as though nothing happened, because S itself is random: it changes from sample to sample. Using the sample to estimate another unknown quantity means our margin of error needs to become larger, and consequently the resulting confidence interval becomes wider.

17.2 Enter the t-Distribution

To account for this extra uncertainty, we use the t-distribution. The confidence interval becomes

$$\boxed{\bar{X} \pm t\frac{S}{\sqrt{n}}} \qquad \text{instead of} \qquad \bar{X} \pm Z\frac{\sigma}{\sqrt{n}}.$$

This is one of the most important distinctions in this chapter:

$$\text{if } \sigma \text{ is known: } \boxed{\bar{X} \pm Z\frac{\sigma}{\sqrt{n}}}; \qquad \text{if } \sigma \text{ is unknown: } \boxed{\bar{X} \pm t\frac{S}{\sqrt{n}}}.$$

The normal distribution is more concentrated in the center, while the t-distribution is flatter and has more probability in its tails. Because the total area under any probability density must still equal one, shifting more area into the tails means there is less mass near the center. We want heavier tails because, when we estimate σ instead of knowing it exactly, we should acknowledge that extreme errors are somewhat more plausible. The heavier-tailed t-distribution does exactly that: it produces a larger critical value than the corresponding Z critical value, especially for small samples, and that larger critical value makes the confidence interval wider. The t-distribution is essentially a penalty for not knowing the true population standard deviation.

17.3 Degrees of Freedom Appear Again

The appropriate t-distribution depends on the number of **degrees of freedom**. For a one-sample mean,

$$df = n - 1.$$

This is the same degree-of-freedom logic discussed earlier for the sample variance: one parameter, the sample mean, has been estimated from the data, so one degree of freedom is lost. As the sample size increases, $df = n - 1$ becomes larger, and the t-distribution gradually approaches the normal distribution. In the limit, with infinitely many degrees of freedom, the t-distribution essentially becomes the normal distribution — the normal distribution is the limiting form of the t-distribution as degrees of freedom become very large. This makes intuitive sense: with a very large sample, the estimate S of σ becomes highly accurate, so there is less need for the extra t-distribution correction.

A subtle but important point concerns $n - 1$. When we calculate S, we use $n - 1$. But in the standard-error expression we still use $S/\sqrt{n}$, not $S/\sqrt{n-1}$. These two appearances of sample size come from different pieces of theory: the $n - 1$ inside S corrects the estimation of the population variance, while the $\sqrt{n}$ in the denominator of the standard error comes from the variance of the sample mean,

$$\mathrm{Var}(\bar{X}) = \frac{\sigma^2}{n}.$$

So even when σ is unknown and replaced by S, the standard-error structure remains $S/\sqrt{n}$. These are different corrections and should not be mixed.

17.4 Known Sigma versus Unknown Sigma, and Proportions

The decision rule can be summarized very simply.

Case 1: Population standard deviation known. Use Z:

$$\bar{X} \pm Z_{\alpha/2}\frac{\sigma}{\sqrt{n}}.$$

Case 2: Population standard deviation unknown. Estimate it using the sample standard deviation S, calculated using $n - 1$, and use t:

$$\bar{X} \pm t_{\alpha/2,n-1}\frac{S}{\sqrt{n}}.$$

A third case, briefly mentioned, is estimating a **population proportion**: the procedure uses the Z-score rather than the t-score. So the summary table has three cases: known population standard deviation uses Z; unknown population standard deviation uses t; and a population proportion uses Z. The detailed treatment of proportions is deferred to a later chapter.

17.5 Different Confidence Levels

The critical value depends on the confidence level. For a 95 percent confidence interval, we used $Z = 1.96$. For a 90 percent confidence interval, 90 percent of the area should be in the center, leaving 10% outside; with a symmetric interval, the two tails each contain 5%. To obtain the positive critical value, we look for cumulative probability 0.95, which gives $Z \approx 1.645$. The general lesson is not to memorize every possible Z-value, but to understand how the confidence level determines the tail probability.

This reinforces another tradeoff: for the same sample, a 90 percent confidence interval is narrower than a 95 percent interval, and a 95 percent interval is narrower than a 99 percent interval. Higher confidence requires capturing more of the probability distribution, which means moving farther into the tails, increasing the critical value and therefore the margin of error:

$$\text{higher confidence} \quad \Longrightarrow \quad \text{wider interval.}$$

There is no contradiction here — greater confidence comes at the price of lower precision.

The standard intervals used in this chapter are symmetric, $\bar{X} - MOE$ to $\bar{X} + MOE$, which corresponds to treating overestimation and underestimation equally. But in some applications we may care much more about one direction — for instance, underestimating demand may be much more costly than overestimating it. In such cases, one could construct a **one-sided confidence bound** or otherwise allocate tail probability asymmetrically. The symmetric interval is a modeling choice rather than an absolute necessity.

17.6 Reversing the Problem: How Much Data Do We Need?

Until now, the sample size n was given, and we asked how precise our estimate is. Now we reverse the question: before collecting data, suppose we decide how precise we want the estimate to be, and ask how many observations must we collect. This is simply reverse engineering the margin-of-error formula.

Consider the known-σ case. Margin of error is

$$MOE = Z\frac{\sigma}{\sqrt{n}}.$$

Suppose the maximum acceptable margin of error is E. Then set

$$E = Z\frac{\sigma}{\sqrt{n}}.$$

Solving for n: multiply both sides by $\sqrt{n}$ to get $E\sqrt{n} = Z\sigma$, so $\sqrt{n} = Z\sigma/E$. Squaring both sides,

$$\boxed{n = \left(\frac{Z\sigma}{E}\right)^2}.$$

This equation gives the minimum sample size required to reach the target margin of error.

17.7 The Salary Example, Revisited

The target margin of error is 10,000. For a 95 percent confidence interval, $Z = 1.96$. The population standard deviation is approximately 47,506.15. So,

$$n = \left(\frac{1.96 \times 47{,}506.15}{10{,}000}\right)^2.$$

The calculation gives approximately 86.69. But we cannot collect 86.69 observations, and if the requirement is *at least* the desired precision, we must always round upward. Therefore,

$$n = 87.$$

At least 87 observations are required to obtain a margin of error no larger than 10,000 under the assumptions of this example. Even if the result were 86.01, we would still need 87 observations — rounding downward would fail to guarantee the targeted precision.

17.8 The Deeper Lesson from the Sample-Size Formula

Consider again

$$n = \left(\frac{Z\sigma}{E}\right)^2.$$

This formula summarizes several important relationships. If the population is more variable, σ is larger, so we need more observations:

$$\sigma \uparrow \quad \Longrightarrow \quad n \uparrow .$$

If we want higher confidence, the critical value Z becomes larger, so we need more observations:

$$\text{confidence} \uparrow \quad \Longrightarrow \quad n \uparrow .$$

If we demand a smaller margin of error E, we need substantially more observations. Most importantly, E is in the denominator and is squared, so if we cut the desired margin of error in half, $E \to E/2$, the required sample size becomes $4n$. This is exactly the square-root relationship discussed earlier, now seen from the reverse direction.

17.9 Connecting the Whole Chapter Sequence

We can now see the complete logical progression across these chapters. We began with a population probability distribution: if we knew that distribution, we could calculate its expected value and variance directly. But in real applications, we usually do not know

the distribution, so we take a random sample. Using that sample, we estimate population quantities: the population mean μ is estimated by $\bar{X}$, and the population variance σ^2 is estimated by $S^2 = \frac{1}{n-1}\sum(X_i - \bar{X})^2$. Random sampling is essential because it allows these sample statistics to meaningfully represent the population.

The sample mean itself is random: its expected value is $E[\bar{X}] = \mu$, and its variance is $\text{Var}(\bar{X}) = \sigma^2/n$, so its standard error is $\sigma/\sqrt{n}$. The Central Limit Theorem tells us that the sampling distribution of $\bar{X}$ becomes approximately normal as sample size grows, which enables us to turn a point estimate into a confidence interval: $\bar{X} \pm Z\,\sigma/\sqrt{n}$ if σ is known, or $\bar{X} \pm t\,S/\sqrt{n}$ if σ is unknown. Finally, if we decide in advance how much estimation error we are willing to tolerate, we reverse the margin-of-error equation to determine how much data we need.

The Most Important Formulas to Remember

- Sample mean: $\bar{X} = \dfrac{1}{n}\sum_{i=1}^{n} X_i$.
- Sample variance: $S^2 = \dfrac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2$.
- Sampling distribution of the sample mean: $E[\bar{X}] = \mu$, $\text{Var}(\bar{X}) = \dfrac{\sigma^2}{n}$.
- Standard error: $SE(\bar{X}) = \dfrac{\sigma}{\sqrt{n}}$.
- Known population standard deviation: $CI = \bar{X} \pm Z\dfrac{\sigma}{\sqrt{n}}$.
- Unknown population standard deviation: $CI = \bar{X} \pm t\dfrac{S}{\sqrt{n}}$.
- Sample size for a target margin of error E, when σ is known: $n = \left(\dfrac{Z\sigma}{E}\right)^2$.

This completes the chapter sequence on random variables, distributions, and statistical estimation. The next part turns to hypothesis testing, which builds directly on the sampling-distribution and confidence-interval logic developed here.

Part IV

Simulation, Monte Carlo Methods, and Optimization with Simulation (Module 4)

18 | Monte Carlo Simulation and the Inverse Transformation Method

18.1 The Central Question of Simulation

Simulation is essentially about generating random samples. Suppose we know a probability distribution; the question is how a computer can actually generate observations from that distribution. This sounds simple when the distribution is something familiar, such as a normal distribution, but the same question appears in far more complicated settings. A large language model, for example, can be viewed as sampling from a complicated probability distribution over possible tokens: given exactly the same prompt twice, the model does not necessarily produce the same answer, because sampling is involved. Random-number generation therefore matters far beyond traditional simulation.

This module develops three methods for generating random observations: **inverse transformation**, **acceptance–rejection**, and **Metropolis**. The important progression is that each method removes an important limitation of the previous one, so rather than memorizing three unrelated algorithms, it is useful to think of them as an evolutionary sequence.

18.2 Uniform Random Numbers as the Building Block

The starting point for every method is the uniform distribution between zero and one, written $U \sim \text{Uniform}(0, 1)$. Computers can readily generate pseudo-random numbers that behave approximately like independent draws from this distribution — repeated draws such as $0.17,\ 0.83,\ 0.42,\ 0.006,\ 0.71, \ldots$ with every region of equal length between zero and one equally likely. These are called **pseudo-random numbers**, since the randomness is produced computationally rather than being ideal mathematical randomness.

The natural question is then: suppose what we actually want is not a uniform random variable, but a normal random variable, a triangular random variable, or a discrete variable taking specific values with specified probabilities. How do we turn a uniform random number into the distribution we actually want? That question leads directly to inverse transformation.

18.3 Stretching and Squeezing: An Intuition for Transformation

A useful intuition is to think of the interval from zero to one as a piece of elastic material: some regions can be **stretched** and others **squeezed**. Doing this correctly transforms a

uniform distribution into another probability distribution.

Consider the simplest example, $W = 4U$. Because U ranges between 0 and 1, W ranges between 0 and 4 — the horizontal range has been stretched by a factor of four. But probabilities must still sum to one, so if the original uniform density has height 1 over an interval of length 1, the new density must have height $\frac{1}{4}$ over an interval of length 4. In short: **stretch horizontally → shrink vertically**, which preserves total probability. This idea generalizes: instead of stretching every part of the interval by the same factor, we can stretch different regions by different amounts, and that nonlinear stretching is exactly what allows us to create complicated target distributions.

18.4 From Density to CDF to Inverse CDF

Suppose the target distribution has density g. From that density we obtain its cumulative distribution function (CDF), denoted G, which answers: what is the probability that the random variable is less than or equal to a particular value x? For a normal distribution, for instance, the CDF starts near zero far to the left, equals 0.5 at the mean, and approaches one far to the right. The CDF therefore maps values of X into numbers between zero and one.

Our simulation problem requires exactly the opposite direction: we already possess a number between zero and one and want to turn it into X. Reversing the mapping gives the **inverse CDF**, G^{-1}, which is why the technique is called the **inverse transformation method**.

18.5 The Inverse Transformation Algorithm

The procedure has three steps:

1. Determine the CDF of the target distribution.
2. Invert that CDF.
3. Generate $U \sim \text{Uniform}(0, 1)$ and compute the corresponding value using the inverse CDF.

Symbolically, the central formula is

$$X = F^{-1}(U).$$

For interpretation purposes this is more important than the symbol itself: generate a percentile at random, and then ask what value of X corresponds to that percentile. That is inverse transformation.

18.6 A Discrete Example

Suppose X can take values $0, 1, 2, 3, 4$ with probabilities 0.2, 0.4, 0.2, 0.1, 0.1. The cumulative probabilities are therefore 0.2, 0.6, 0.8, 0.9, 1. Turning this into intervals on the uniform $(0, 1)$ scale:

- If $U \in (0, 0.2)$, output 0.
- If $U \in (0.2, 0.6)$, output 1.
- If $U \in (0.6, 0.8)$, output 2.
- If $U \in (0.8, 0.9)$, output 3.
- If $U \in (0.9, 1)$, output 4.

The interval corresponding to the value 1 has length $0.6 - 0.2 = 0.4$, so a uniform random number lands in it with probability 0.4 — exactly the desired probability of $X = 1$. Similarly, the interval corresponding to 3 has length 0.1, so 3 appears with probability 0.1. This is the essence of the method.

18.7 A Second Discrete Example

Consider a random variable taking values $10, 15, 20, 25$ with probabilities 0.1, 0.7, 0.1, 0.1. The inverse transformation mapping is:

- $U \in (0, 0.1) \Rightarrow 10$
- $U \in (0.1, 0.8) \Rightarrow 15$
- $U \in (0.8, 0.9) \Rightarrow 20$
- $U \in (0.9, 1) \Rightarrow 25$

The value 15 receives such a large interval because it should occur seventy percent of the time, and the interval from 0.1 to 0.8 occupies seventy percent of the uniform $(0, 1)$ range. This is **squeezing an entire region of uniform numbers into the same output value** — an interpretation worth remembering because it provides intuition rather than just an algorithm.

18.8 Continuous Distributions: The Normal Example

Now consider generating a normal random variable with mean 100 and standard deviation 20. Generate $U \sim \text{Uniform}(0, 1)$.

- If $U = 0.5$: the 50th percentile of a normal distribution is its mean, so $X = 100$.
- If $U = 0.7$: the standard normal value with cumulative probability approximately 0.7 is $Z \approx 0.52$–0.53, so $X \approx 100 + 0.52(20) \approx 110$.
- If $U = 0.99$: the corresponding $Z \approx 2.33$, so $X \approx 100 + 2.33(20) \approx 146.6$.

The exact numerical value is less important than the procedure: we are asking *which normal value has cumulative probability equal to* U, using the cumulative normal table (or Φ^{-1}). As U approaches one, the transformed value grows increasingly large; as U approaches zero, it moves further into the negative tail. This is the stretching phenomenon again: the middle of the uniform distribution is stretched relatively little, while the extreme regions near 0 and 1 are stretched enormously.

18.9 The Triangular Distribution

Consider a triangular distribution with minimum 20, mode 23, and maximum 26. Because half of the triangular distribution lies below 23, $U = 0.5$ maps to $X = 23$. But suppose $U = 0.25$: we now need the value X for which the area under the triangular density from 20 to X equals 0.25, which becomes a geometry problem.

An important observation is that X is **not** simply halfway between 20 and 23. The density is increasing as X moves from 20 toward 23, so if we stopped at the midpoint 21.5, both the base and the height of the resulting small triangle would be reduced by one-half, so its area is reduced by $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$. The cumulative probability at the midpoint is therefore only one quarter of the probability contained in the entire left half of the distribution. This illustrates a general principle: **percentiles are determined by probability area, not by geometric distance along the x-axis.**

Key Takeaways

- Simulation begins with uniform $(0, 1)$ pseudo-random numbers.
- The CDF converts observations into cumulative probabilities; the inverse CDF does the opposite, converting cumulative probabilities into observations: $X = F^{-1}(U)$.
- Inverse transformation can be understood as stretching and squeezing the uniform distribution until it has the shape of the target distribution.

18.10 The Limitation of Inverse Transformation

To use inverse transformation we need enough information about the target distribution to obtain its CDF and then invert it. In many realistic problems this is difficult, because the target density generally needs to be **normalized** — its total probability must equal one. In a gigantic state space involving sentences, images, or other high-dimensional objects, we might only know that "object A looks twice as plausible as object B." That is a **score**, not a probability, and converting all such scores into a normalized distribution can be computationally prohibitive. Without a normalized distribution, constructing and inverting the CDF becomes impractical. This motivates the second method developed in this module: **acceptance–rejection sampling**, whose conceptual advantage is that we can often work with **unnormalized scores** rather than requiring the complete normalized probability distribution.

19 | Acceptance–Rejection Sampling

19.1 Motivation: Working with Unnormalized Scores

Inverse transformation is conceptually elegant, but it generally requires a properly normalized probability distribution from which to construct and invert a CDF. In complicated real-world situations normalization can be extremely difficult: with an enormous number of possible sentences or images, we might have a function assigning a score to each outcome without any convenient way to compute the total score over every possible outcome. Acceptance–rejection is motivated as a method that operates directly on scores, without requiring that normalization constant to be calculated.

19.2 Scores versus Probabilities

The distinction between a **score** and a **probability** is central. Suppose four outcomes have scores $5, 10, 5, 5$. These do not sum to one, so they are not probabilities; dividing each by their sum (25) gives the normalized probabilities 0.20, 0.40, 0.20, 0.20. Acceptance–rejection asks whether that normalization step can be avoided: if all we need is to generate observations with frequencies proportional to the scores, perhaps we can work with the scores directly. In very large spaces, summing over every possible outcome could be computationally enormous, so acceptance–rejection instead uses **relative** scores.

19.3 The Acceptance Ratio and the Mode

Acceptance–rejection requires knowing the **mode** — the outcome with the highest score — which serves as a benchmark. For any candidate outcome x,

$$\text{acceptance probability} = \frac{\text{score}(x)}{\text{score}(\text{mode})}.$$

Because the mode has the maximum score, this ratio is always between zero and one, making it a valid acceptance probability. Crucially, the maximum score itself does not have to be normalized — it could be 0.4, or 10, or any other scale; only the *relative* scores matter.

Since the mode's acceptance probability is score/score $= 1$, **the mode is always accepted**, which makes intuitive sense: it is supposed to be the most frequently occurring outcome. A candidate with score 0.2 relative to a mode of 0.4 has acceptance probability 0.5; a candidate with score 0.1 has acceptance probability 0.25. The rejection mechanism changes the frequencies of the initially generated observations until their relative frequencies match the target scores.

19.4 The Two-Stage Sampling Process

Acceptance–rejection is a **two-stage generation process**. In the first stage we generate a candidate, which is not necessarily the final output. In the second stage we decide whether to accept that candidate: if accepted, it becomes part of the sample; if rejected, we discard it and generate another candidate. Verbally: generate something, inspect it, and if it satisfies the acceptance criterion let it pass — otherwise reject it and regenerate. That is the fundamental architecture of acceptance–rejection sampling.

19.5 Why Rejection Reproduces the Target Distribution

Suppose a first-stage generator produces outcomes $1, 2, 3, 4$ uniformly, and the target scores are 0.20, 0.15, 0.25, 0.40 respectively, so value 4 is the mode. The acceptance probabilities are

$$\frac{0.20}{0.40} = 0.50, \qquad \frac{0.15}{0.40} = 0.375, \qquad \frac{0.25}{0.40} = 0.625, \qquad \frac{0.40}{0.40} = 1.$$

These are *acceptance* probabilities, not the probabilities of the final distribution — they need not sum to one. Imagine the first-stage generator produces 100 observations of each value. Applying the acceptance rates leaves approximately 50, 37.5, 62.5, and 100 accepted observations respectively, out of 250 total. Dividing by 250 gives approximately 0.20, 0.15, 0.25, 0.40 — exactly the target probabilities. So although the initial generator is uniform, the rejection mechanism reshapes the distribution into the target one.

19.6 Why Normalization Disappears

Suppose instead of probabilities we had raw scores $20, 15, 25, 40$. The acceptance probabilities are still 0.50, 0.375, 0.625, 1 — identical to before. If every score were multiplied by one million, nothing would change, because the common scaling factor cancels in every ratio. This is why unnormalized scores are sufficient: models can score objects, sentences, or other outcomes even though those scores do not naturally sum to one.

19.7 Sampling from the Triangular Score Function

Return to the triangular distribution with minimum 20, mode 23, maximum 26. Where inverse transformation required calculating areas under the triangle, acceptance–rejection offers another way: assign the triangular function height 1 at its peak. The resulting triangle has base 6 and height 1, so its area is $\frac{1}{2}(6)(1) = 3$ — **not** normalized (a proper density would need area one), but acceptance–rejection does not care; the triangle can still be used directly as a score function.

First generate a candidate uniformly between 20 and 26. Suppose the candidate is 23 (the mode): its score is 1, so the acceptance probability is $1/1 = 1$, i.e. always accept. Suppose the candidate is 21.5, halfway between 20 and 23: its height under the triangle is 0.5, so the acceptance probability is $0.5/1 = 0.5$ — accept with probability 50%. Similarly, a candidate of 24.5, halfway between 23 and 26 on the descending side, also has score 0.5 and is accepted with probability 50%. Notice how much simpler this is than solving

the inverse-CDF geometry problem: we only need the height of the score function at the candidate relative to its maximum height, not total areas.

19.8 Implementing "Accept with Probability p"

Given an acceptance probability for a candidate, how do we actually implement "accept with probability p"? Generate a second, independent uniform random number $V \sim \text{Uniform}(0, 1)$ and apply the rule: accept if $V \leq p$, otherwise reject. For an acceptance probability of 0.375, accept if $V \leq 0.375$; for 0.625, accept if $V \leq 0.625$. For the mode, whose acceptance probability equals one, every possible $V \in (0, 1)$ satisfies $V \leq 1$, so the mode is always accepted. This gives a general implementation rule for any acceptance probability.

19.9 The Complete Acceptance–Rejection Algorithm

For a discrete example, the process is:

1. Generate a candidate uniformly from the possible outcomes.
2. Determine the candidate's acceptance probability, score(candidate)/score(mode).
3. Generate an independent $V \sim \text{Uniform}(0, 1)$.
4. If V falls inside the acceptance region, accept the candidate; otherwise reject it and generate another candidate.
5. Repeat until enough accepted observations have been obtained.

A rejected observation is simply discarded, not converted into something else; the algorithm then tries again. For example, if candidate 2 has acceptance probability 0.375 and the second uniform draw is $0.70 > 0.375$, candidate 2 is rejected and a new candidate is generated. If the next candidate is 4, with acceptance probability 1, it is accepted automatically. This repeats until the desired sample size is reached.

19.10 Exam Guidance

For an acceptance–rejection question involving discrete outcomes, it is generally not necessary to simulate thousands of observations. The critical answer is the **acceptance table** — for example, value 1 has acceptance probability 0.50, value 2 has 0.375, value 3 has 0.625, and value 4 has 1 — together with a clear explanation of the candidate-generation step. Presenting the acceptance rates and the procedure is enough for exam purposes; actually programming the sampling algorithm is more relevant to a computational assignment.

19.11 A Continuous Example: Sampling Up to an Unknown Normalization Constant

A particularly important continuous example: suppose the target density has the form

$$p(x) = C\, f(x), \qquad -2 \le x \le 2,$$

where f is the standard normal density and C is an unknown normalization constant. At first this looks problematic, since C is unknown. But acceptance–rejection does not require knowing it, because when we form a ratio of scores, C cancels. The mode of the standard normal distribution is zero, so zero has the maximum score and is accepted with probability one; other candidates are accepted relative to the standard normal density evaluated at zero. This is an excellent illustration of why acceptance–rejection is useful when the target density is known only **up to a normalization constant**.

19.12 Generating Uniform Candidates on an Arbitrary Interval

To sample candidates uniformly between -2 and 2 from a base $U \sim \text{Uniform}(0, 1)$, use

$$X = 4U - 2.$$

Multiplying U by 4 changes the range from $(0, 1)$ to $(0, 4)$; subtracting 2 shifts that interval to $(-2, 2)$. More generally, to obtain a uniform random variable on (a, b), use

$$X = a + (b - a)U.$$

The guiding principle is conceptual rather than a formula to memorize: **stretch first, then shift.**

19.13 Comparing Inverse Transformation with Acceptance–Rejection

Inverse transformation is direct: generate U and immediately transform it into X. If the inverse CDF is easy to calculate, the method is elegant and efficient, but it requires the CDF and its inverse, which can be difficult to obtain. Acceptance–rejection avoids that requirement, needing only relative scores and the maximum score — but at a price: many candidates may be rejected, wasting computational effort. If the target distribution is very concentrated relative to the proposal distribution, most generated candidates might be rejected. There is always a tradeoff between the two methods.

19.14 The Remaining Weakness of Acceptance–Rejection

Acceptance–rejection removes the need to calculate the normalization constant, but it still requires knowing something important: **the mode and its score**, since the

maximum score is the denominator of the acceptance ratio. If the state space is incredibly complicated — for example, if finding the globally most likely sentence, image, or configuration is itself extremely difficult — then acceptance–rejection still has a limitation. Finding the mode of a distribution can itself become very challenging in real-world problems, which motivates **Metropolis** as a more generic technique that removes this requirement.

Key Takeaways

- Acceptance–rejection generates candidates easily and then selectively filters them according to their relative scores: acceptance probability = score(x)/score(mode).
- The mode is accepted with probability one; lower-score outcomes are increasingly filtered out.
- Advantage over inverse transformation: no normalization constant or inverse CDF is required.
- Remaining disadvantage: the mode (maximum score) must still be known, and rejection can waste samples.

20 | Markov Chains and the Metropolis Algorithm

20.1 Where Metropolis Fits

There are three major random-number-generation techniques in this module. The first is the **transformation technique**: start with a uniform random variable, which computers generate easily, and transform it into a random variable with the desired target distribution. The second is **acceptance–rejection sampling**: generate random candidates and use another random mechanism to decide whether each proposal should be accepted or rejected, selectively reshaping the proposal distribution into the target distribution. The third technique, and the principal topic of this chapter, is the **Metropolis algorithm**, which belongs to the broader family of methods called **Markov Chain Monte Carlo (MCMC)**. Other MCMC methods exist, such as Gibbs sampling, but this module focuses on Metropolis.

The important conceptual difference is that Metropolis does not attempt to generate an independent observation from the target distribution in one step. Instead, it constructs a **Markov chain** whose long-run behavior corresponds to the target distribution.

20.2 What Is a Markov Chain?

Suppose the target random variable can take ten values, $1, 2, \ldots, 10$, thought of as ten possible states, and suppose we are currently at state 3. The defining property of a Markov chain is that the probability distribution of the **next state depends only on the current state**, not on the complete history of the process: it does not matter whether we arrived at state 3 from state 2, from state 4, or were already there in the previous iteration. All that matters is that we are currently at state 3. A useful analogy is a person wandering between locations who does not remember how they arrived at their present location; their next movement is determined only by where they currently are. This is the **Markov property**.

20.3 What Are We Trying to Accomplish?

Suppose the desired probabilities of states 1 through 10 are unequal — for example, $P(1) = 0.11$, $P(2) = 0.12$, $P(3) = 0.09$, and so forth. If we simply generated integers uniformly between 1 and 10, every state would appear approximately 10% of the time, which is not our target. Instead, we want to construct a random walk such that, after running it for a sufficiently long time, the fraction of time spent in each state approaches its target probability: if $P(1) = 0.11$, we want approximately 11% of observations to be

state 1, and so on. The objective is not to obtain the correct probability immediately, but to construct transition rules whose **long-run visitation frequencies** reproduce the target distribution. This is the central idea of MCMC.

20.4 Designing the Random Movement

Consider an interior state such as state 3. A simple local movement rule allows the chain to move left to state 2, stay at state 3, or move right to state 4. If these three proposed movements were treated equally (each with probability $\frac{1}{3}$) and simply accepted, the resulting long-run distribution would tend toward something close to uniform. We therefore need a mechanism that encourages movement toward states with higher target scores and discourages movement toward states with lower scores — the **Metropolis acceptance probability**.

20.5 The Metropolis Acceptance Ratio

Suppose we are currently at state i and propose moving to state j. The acceptance probability is based on the ratio P_j/P_i. But this ratio might exceed one, and since probabilities cannot exceed one, we clip the ratio at one:

$$\alpha(i \to j) = \min\left(1,\ \frac{P_j}{P_i}\right).$$

This is perhaps the single most important formula of the chapter. For a proposed move from state 3 to state 4, the acceptance probability is $\min(1, P_4/P_3)$, and the overall probability of actually moving from 3 to 4 is

$$\frac{1}{3} \times \min\left(1, \frac{P_4}{P_3}\right),$$

because two things must happen: we must first propose the rightward move (probability $\frac{1}{3}$), and then, conditional on proposing it, we must accept it. The multiplication rule gives the total transition probability. The same reasoning gives the probability of moving from 3 to 2 as $\frac{1}{3} \times \min(1, P_2/P_3)$.

20.6 Why the Ratio Works

If the proposed state has a higher score than the current state, $P_j/P_i > 1$, and after clipping the acceptance probability becomes one: moves toward a higher-score state are accepted automatically. If the proposed state has a lower score, the ratio is below one and the move is accepted only probabilistically — for example, a ratio of 0.8 means we accept the move with probability 0.8. This creates a systematic tendency toward high-score states without completely preventing movement toward lower-score states.

That last point is important: Metropolis does *not* simply move uphill all the time. If it always moved toward the highest-probability state, the chain would eventually become trapped there. Instead, Metropolis still permits downward moves, merely making them less likely, so the process continues exploring the entire state space while spending proportionally more time in high-probability regions — intuitively, directing more "traffic" toward popular states.

20.7 The Probability of Staying

Once the probabilities of moving left and moving right are calculated, the easiest way to calculate the probability of remaining at the current state is to use the fact that probabilities must sum to one:

$$P(\text{stay at } 3) = 1 - P(\text{move left}) - P(\text{move right}).$$

This staying probability includes several possibilities: we might explicitly propose staying, or we might propose moving left or right but reject the proposal. All of those outcomes leave the Markov chain at the current state. For exam purposes, the easiest method is simply to calculate all outgoing transition probabilities and assign the remaining probability to staying in the current state.

20.8 Boundary States

Interior states such as 3 are straightforward because they have neighbors on both sides, but state 1 and state 10 are different: state 1 cannot move left to 0, and state 10 cannot move right to 11. At state 1, only two proposals are possible — stay at 1, or move to 2 — so the proposal probability becomes $\frac{1}{2}$ rather than $\frac{1}{3}$. The transition probability from 1 to 2 is consequently

$$\frac{1}{2} \times \min\left(1, \frac{P_2}{P_1}\right),$$

and the probability of remaining at 1 is one minus that quantity. State 10 is handled symmetrically.

20.9 Why Normalization Constants Cancel

So far $P_1, P_2, P_3, \ldots$ have been described as probabilities, but in real applications they do **not** actually have to be normalized probabilities. Suppose instead a model gives scores $F(1) = 100$, $F(2) = 200$, $F(3) = 250, \ldots$ that do not need to sum to one. If the true probability has the form $P(x) = F(x)/Z$ for an unknown normalization constant Z, then

$$\frac{P_j}{P_i} = \frac{F(j)/Z}{F(i)/Z} = \frac{F(j)}{F(i)}.$$

The Z terms cancel, so we never need to calculate the normalization constant. This is an extraordinarily important property: Metropolis only uses *ratios* of scores.

20.10 Why Normalization Can Be Difficult

For a small distribution with ten possible states, normalization is easy: sum all ten scores and divide. But an enormous state space — millions, billions, or vastly more possible configurations, as arise with modern machine-learning models — can make computing the normalization constant computationally impossible. A neural network can produce scores for different outcomes even when explicitly enumerating and normalizing over the entire output space is infeasible. Metropolis can work directly with relative scores

because the unknown common normalization factor cancels in the ratio. This is one of the major conceptual advantages of MCMC over acceptance–rejection, which typically requires additional global information about the target and proposal distributions: if we can evaluate a score proportional to the target density, we can potentially construct a Metropolis sampler.

20.11 Burn-In and Convergence

Suppose the chain is initialized at state 1. Initially, observations may depend strongly on that arbitrary starting location, but as the chain continues moving it gradually loses information about where it began, eventually approaching the target stationary distribution. The early observations are therefore often discarded — this is commonly called the **burn-in period**. Allowing only local movements (left, right, or stay) may require many iterations before the process settles into equilibrium behavior; allowing longer-distance proposals can speed up movement through the state space, although it makes the mechanism more complicated. There is a tradeoff: a simple proposal mechanism is easier to implement but may mix slowly, while a richer proposal mechanism explores more quickly but requires more computational effort. This module emphasizes the simpler neighboring-state construction.

20.12 Does the Initial State Matter?

In the long run, assuming the chain has the appropriate properties, the initial state should not determine the stationary behavior — the chain could be started at state 1, 5, or 10, and after enough transitions it effectively "forgets" its starting location. What ultimately matters is the transition mechanism, not the arbitrary initialization; this is another way to understand burn-in, since early observations contain information about initialization while later observations increasingly reflect the stationary behavior of the chain.

Key Takeaways

- Metropolis converts relative scores into a random walk whose long-run visitation frequencies reproduce the target distribution.
- Propose random movements through the state space, then accept them with probability $\min(1, \text{proposed-state score}/\text{current-state score})$.
- Higher-score states attract more traffic; lower-score states are still visited, but less frequently.
- Because Metropolis works with ratios, unknown normalization constants cancel, making it exceptionally useful for complicated probability distributions.

21 | Implementing Metropolis and Extending It to Continuous Distributions

21.1 Two Stages: Construction and Generation

It is useful to separate the Metropolis procedure into two stages. In the **preparation stage** we calculate the state-transition probabilities. In the **generation stage** we repeatedly generate uniform random numbers and use those transition probabilities to decide where the Markov chain moves. For a ten-state example, the first and last states have two possible transitions while the eight interior states have three each, giving a transition table with 28 entries. Manually producing a large transition table is not practical — with 1,000 states, the probabilities would naturally be generated through a loop in code. For exam purposes, the emphasis is on knowing *how* the transition probabilities are constructed, rather than writing the corresponding code.

21.2 From Transition Probabilities to a Random Move

Suppose we are currently in state 1, where the only possibilities are to stay at 1 or move to 2, each with transition probability $\frac{1}{2}$. To implement this, generate $U \sim \text{Uniform}(0, 1)$ and partition the interval from 0 to 1 according to the transition probabilities: if U falls in the first half, move from 1 to 2; if it falls in the second half, stay at 1. For example, if $U = 0.07$, it falls in the first region and the chain moves from state 1 to state 2; if the next draw is $U = 0.56$, it falls in the second region and the chain stays at 1. The transition probabilities determine **how wide each region should be**; the uniform random number determines **which transition actually occurs** — the same general simulation principle used throughout this module: map intervals of a uniform random variable into particular outcomes.

21.3 Interior States Have Three Regions

For an interior state such as state 2, there are three possible outcomes: move left to 1, stay at 2, or move right to 3. Suppose these transition probabilities, computed from the Metropolis formulas, are approximately 0.25 and 0.305 (with the remainder assigned to staying). We construct cumulative intervals: the first from 0 to 0.25, the second from 0.25 to about 0.555, and the remainder for the third outcome. Generating $U = 0.09$ falls in the first region, $U = 0.40$ falls in the second, and $U = 0.82$ falls in the third. A single uniform random number therefore determines the next state once the transition probabilities have been translated into cumulative regions.

21.4 Popular States and Long-Run Frequencies

If a state has a relatively high target probability, the transition mechanism will create relatively large probability mass associated with staying there or returning there; as a result, whenever the chain reaches such a "popular" state, it tends to spend more time there, and repeated simulation produces more observations of that state. A less probable state attracts less traffic. After many iterations, these differences in visitation frequencies create the target distribution. This is the underlying mechanism of Metropolis: we never directly instruct the simulation to "generate state 2 exactly 12% of the time." Instead we construct local transition probabilities, and the Markov chain automatically produces the desired global frequencies after sufficiently many iterations.

21.5 The Complete Discrete Metropolis Algorithm

Suppose we have states 1 through K with target scores $f(1), \ldots, f(K)$.

1. Start at any state i.
2. For an interior state, identify the three possible proposals $i-1$, i, and $i+1$.
3. Calculate the actual transition probabilities using the Metropolis score ratios, e.g. the probability of moving right is $\frac{1}{3}\min(1, f(i+1)/f(i))$, and similarly for moving left.
4. Calculate the stay probability as the residual: one minus the other outgoing probabilities.
5. Generate $U \sim \text{Uniform}(0,1)$, divide the interval $(0,1)$ into regions according to the transition probabilities, and observe where U falls to determine the next state.
6. Let the next state become the current state, and repeat many times (on the order of thousands of iterations).

Eventually the Markov chain reaches its stable behavior, and the empirical frequencies with which it visits the states approximate the target distribution.

21.6 Moving to the Continuous Case

Suppose the target distribution is continuous. In the discrete example, moving left or right was straightforward: if we were at state 3, the neighbors were 2 and 4. But suppose X is continuous and we are currently at $X = 21$ — is the next state 21.1? 21.01? 21.001? There is no unique "next" real number, so the discrete left-right-neighbor construction cannot be used literally. The solution is to generate a **continuous random proposal**.

21.7 The Triangular Distribution Example

Consider the same triangular target used earlier: lower endpoint 20, mode 23, upper endpoint 26 (the distribution need not generally be symmetric, though this particular example has the mode in the middle). Suppose the current state is $X = 21$. How should we propose a new state?

Instead of "move one state left" or "move one state right," generate $U \sim \text{Uniform}(-1, 1)$, representing a random movement: if U is positive we move in one direction, if negative the other. The proposed state is

$$X_{\text{proposal}} = X + U.$$

If the current state is 21 and $U \approx -0.104$, then $X_{\text{proposal}} \approx 21 - 0.104 = 20.896$. This is only a **proposal**: the key vocabulary is *current state*, *proposal*, *acceptance probability*, and then *accepted or rejected next state*.

21.8 The Acceptance Rule Is Essentially Unchanged

Once the proposal is generated, calculate the score at the proposed point, $f(X_{\text{proposal}})$, and at the current point, $f(X)$, then compute

$$\alpha = \min\left(1, \frac{f(X_{\text{proposal}})}{f(X)}\right).$$

This is essentially the same formula used in the discrete problem; the only difference is how the proposed state is generated. In the discrete case the proposal is left or right; in the continuous case a continuous random perturbation determines how far in either direction we propose to move.

21.9 Constructing an Unnormalized Score Function for the Triangle

We do not need the normalized triangular probability density — only a function with the appropriate relative shape. From 20 to 23 the density is increasing, so we can use the score $f(x) = x - 20$ for $20 \leq x \leq 23$. After 23 the triangle decreases to zero at 26, so we use $f(x) = 26 - x$ for $23 \leq x \leq 26$. At $x = 23$ both expressions give 3, so the two pieces meet, and the resulting piecewise function has the required triangular shape without needing to be normalized. If the entire score function were multiplied by 100, the acceptance ratio $100f(X_{\text{proposal}})/100f(X)$ would be unchanged, since the 100 cancels — again, Metropolis requires relative scores, not necessarily normalized probabilities.

21.10 A Numerical Acceptance Example

Suppose the current state is $X = 21$ and the proposed state is $X_{\text{proposal}} \approx 20.895$; both lie on the increasing part of the triangle. The score at the current state is $21 - 20 = 1$; the score at the proposed state is approximately $20.895 - 20 = 0.895$. The ratio is $0.895/1 \approx 0.895$, so $\alpha \approx 0.895$: the proposed move is accepted with probability around 89.5%. It is not accepted with probability one because the proposal moved slightly away from the mode, so its score is slightly lower.

Now consider the opposite case: at 21, propose moving to 21.5. Since this moves upward toward the mode at 23, the score at 21.5 exceeds the score at 21, the ratio exceeds one, and after clipping $\alpha = 1$ — the move is accepted automatically. This illustrates

the fundamental Metropolis behavior: moves toward higher-score regions are accepted, while moves toward lower-score regions are accepted with probability less than one, but downward moves remain possible. This prevents the algorithm from degenerating into an optimization procedure that climbs to the mode and stays there; the objective is **sampling**, not optimization.

21.11 Implementing the Accept/Reject Decision

After obtaining α, generate a second independent uniform random number $V \sim \text{Uniform}(0, 1)$. If $V \leq \alpha$, accept the proposal and set $X_{\text{next}} = X_{\text{proposal}}$; if $V > \alpha$, reject it and set $X_{\text{next}} = X$. Repeat the entire procedure — generate another proposal perturbation, calculate a new ratio, accept or reject — many times. The resulting sequence of X values forms the Markov chain.

21.12 Boundary Handling in the Continuous Case

The continuous target exists only between 20 and 26. Near 20, a negative perturbation could propose a value below 20, which lies outside the support of the target distribution (similarly for a proposal above 26). One way to view the algorithm is that points outside the target domain have zero target score and therefore zero acceptance probability, so an invalid proposal is simply not accepted; the chain remains at the current legitimate value and proceeds to the next iteration. Proposals can also be explicitly restricted or clipped near the boundaries as an implementation detail.

21.13 Discrete and Continuous Metropolis Are the Same Idea

At first glance the discrete and continuous versions look different, but conceptually they are almost identical. For the discrete case: current state, propose a neighboring state, compare proposed and current scores, accept according to the ratio, otherwise stay. For the continuous case: current value, generate a random perturbation, create a proposed value, compare proposed and current scores, accept according to the ratio, otherwise stay. The critical formula in both cases remains $\alpha = \min(1, \text{proposed score/current score})$; the main difference is only the **proposal mechanism** — a naturally defined "neighbor" in discrete space versus a randomly generated perturbation in continuous space.

21.14 Two Kinds of Randomness

There are two different kinds of randomness inside the continuous Metropolis algorithm. The first randomness creates the **proposal** (e.g. $U \sim \text{Uniform}(-1, 1)$), answering "where should I consider moving?" The second randomness determines **acceptance** (e.g. $V \sim \text{Uniform}(0, 1)$), answering "given that proposal, should I actually move there?" This proposal-versus-acceptance distinction is one of the most important ways to understand MCMC: the proposal mechanism explores, and the acceptance mechanism reshapes that exploration so that the resulting long-run sample has the target distribution.

21.15 The Complete Continuous Metropolis Algorithm

A compact, exam-ready statement of the algorithm:

1. Start with an initial state X within the support of the target distribution.
2. Generate a random proposal perturbation U (e.g. uniform on $(-1, 1)$) and set $X_{\text{proposal}} = X + U$.
3. Evaluate the target score at the proposed and current states, and compute $\alpha = \min(1, f(X_{\text{proposal}})/f(X))$.
4. Generate $V \sim \text{Uniform}(0, 1)$; if $V \leq \alpha$, accept the proposed state, otherwise remain at the current state.
5. Repeat many times, discarding the initial burn-in observations if necessary.

The remaining states form a sample approximately distributed according to the target distribution.

Key Takeaways

- Metropolis has two layers: generate a candidate move, then decide whether to accept it using a target-score ratio.
- For discrete variables, candidate moves are neighboring states; for continuous variables, candidate moves are generated by adding continuous random noise.
- In both cases, repeated accepted and rejected movements create a Markov chain whose long-run frequencies reproduce the target distribution.
- We can generate complicated random variables without explicitly calculating their normalized probability distribution — we only need a way to compare the relative target score of two states.

22 | Optimization with Simulation: The Hotel Overbooking Problem

22.1 What Does "Optimization with Simulation" Mean?

Simulation does not have to be the end of an analysis; it can be used inside an optimization problem. Optimization with simulation moves repeatedly between two stages: a **simulation** stage, in which we generate random outcomes, and an **optimization** stage, in which we evaluate a decision based on those random outcomes and compare alternative decisions. The overall logic is: choose a decision, simulate its uncertain consequences, estimate its expected performance, change the decision, simulate again, compare, and continue until the best decision is identified. This connects the optimization material developed earlier in the course with the simulation techniques of this module.

22.2 The Hotel Overbooking Example

Let's consider a hotel that has 100 rooms. Suppose that each reservation generates revenue of \$150. If a customer actually arrives and occupies a room, the hotel incurs a preparation/cleaning cost of \$30. Customers have a 5% probability of not showing up, so the probability that a customer *does* show up is 95%. The customer has already paid, and a late no-show does not receive a refund. This creates an opportunity: if some customers fail to arrive, the hotel could accept more than 100 reservations while still having no more than 100 people physically arrive — the economic logic behind **overbooking**.

22.3 Why Overbooking Can Increase Profit

If the hotel accepts exactly 100 reservations, roughly 5% of customers will fail to arrive, leaving empty rooms even though every available room was technically sold (e.g. only 95 customers arrive, leaving 5 rooms unused, even though the hotel still received the booking revenue). Because some customers will probably not arrive, accepting 101, 102, or 103 reservations can reduce unused physical capacity and initially increase expected profit. But the hotel cannot keep increasing reservations indefinitely: eventually too many customers will arrive, and when more than 100 customers show up, some cannot be accommodated, creating an **overbooking penalty** of \$200 per displaced customer. Expected profit as a function of the booking limit should therefore initially increase but eventually decline once the expected penalties dominate the benefit of additional reservations. The optimization problem is to identify that turning point.

22.4 The Decision Variable

Let N denote the number of reservations the hotel accepts. Since the hotel has 100 rooms, the interesting values of N begin at 100: we can evaluate $N = 100, 101, 102, 103, \ldots$. For each value of N we simulate the number of customers who actually show up, calculate the corresponding profit, and after many simulations estimate the expected profit associated with that booking limit. Comparing expected profit across booking limits, we expect it to reach a maximum and then begin to decline; that value of N is the optimal reservation limit under the model's assumptions.

22.5 The Random Variable to Simulate

Suppose the hotel accepts 101 reservations. The uncertain quantity is the **number of customers who show up**, X, taking values $0, 1, 2, \ldots, 101$. Since each reservation holder independently shows up with probability 0.95, X follows a binomial distribution: the number of successes among 101 independent trials. Any of the three simulation techniques developed in this module — transformation, acceptance–rejection, or Metropolis — can in principle be used to simulate X. Using the Metropolis framework as an illustration: if the current state is 3, the neighboring states are 2, 3, and 4, and calculating the transition probability from 3 to 4 requires the ratio $P(X{=}4)/P(X{=}3)$.

22.6 The Binomial Probabilities and Why Their Ratio Simplifies

The probability that exactly 4 of 101 customers show up is

$$P_4 = \binom{101}{4}(0.95)^4(0.05)^{97},$$

and the probability that exactly 3 show up is

$$P_3 = \binom{101}{3}(0.95)^3(0.05)^{98}.$$

The Metropolis acceptance ratio for moving from state 3 to state 4 uses P_4/P_3, and much of this complicated expression cancels: the large factorial terms and large powers of 0.95 and 0.05 mostly cancel, simplifying the ratio to approximately

$$\frac{P_4}{P_3} \approx \frac{98}{4} \times \frac{0.95}{0.05}.$$

The exact simplification matters less than the conceptual lesson: even when individual probabilities look extremely complicated, the **ratio of neighboring probabilities can be simple**. This is exactly why Metropolis is attractive — we often do not need to calculate every probability in full, only enough information to compare neighboring states.

22.7 Choosing a Simulation Technique

The hotel problem does not force the use of any single method; the transformation technique, acceptance–rejection, and Metropolis are all valid choices, and the easiest one to explain and implement is acceptable. A textbook solution might use the transformation technique for the binomial distribution, but Metropolis can be easier to formulate because it avoids explicitly constructing the cumulative distribution function. The general principle — producing a valid simulator for the random number of show-ups — matters more than matching one particular computational method.

22.8 Simulation Is Only Half the Problem

Successfully simulating the number of show-ups is not sufficient: the random-number generator gives X, but the business question concerns **profit**. A second component is required: a profit function mapping the simulated number of show-ups into money. Developing only the random-number simulator represents only part of the answer; it is equally necessary to show how to calculate revenue, cost, and profit in each possible state.

22.9 The Profit Function

Let the hotel accept N reservations and let X be the number of customers who actually arrive.

Case 1: $X \leq 100$**.** The hotel can accommodate everybody. Revenue is $150N$ (all N reservations were sold and paid for, including no-shows), and the operating/cleaning cost is $30X$ (only actual arrivals generate this cost). Hence

$$\text{Profit} = 150N - 30X, \qquad X \leq 100.$$

Case 2: $X > 100$**.** The hotel has only 100 rooms, so it can serve only 100 customers. Revenue remains $150N$, and the room preparation cost is limited to 100 occupied rooms, $30(100)$. The number of customers who cannot be accommodated is $X - 100$, each generating a penalty of \$200, giving a penalty cost of $200(X - 100)$. Hence

$$\text{Profit} = 150N - 30(100) - 200(X - 100), \qquad X > 100.$$

This piecewise structure is fundamental because the economics change exactly when physical capacity is exceeded: below capacity, an additional arrival only creates the \$30 operating cost, while above capacity the hotel cannot serve another arrival and instead incurs the \$200 overbooking penalty.

22.10 One Simulation Replication

Suppose $N = 101$. First simulate the number of show-ups X. If the simulator produces $X = 97$ (below 100), use the first branch:

$$\text{Profit} = 150(101) - 30(97).$$

That gives one simulated profit observation. Repeating the random-number generation, suppose the next draw produces $X = 101$; capacity is now exceeded by one customer, so the second branch applies:

$$\text{Profit} = 150(101) - 30(100) - 200(1).$$

That gives another simulated profit observation, and the process repeats many times.

22.11 Estimating Expected Profit

Suppose we generate 10,000 simulated values of X, calculate the corresponding profit for each, and average those 10,000 simulated profits. That sample average estimates the **expected profit** associated with booking limit N:

$$\widehat{E[\text{Profit} \mid N]} = \frac{1}{10{,}000} \sum_{k=1}^{10{,}000} \text{Profit}_k.$$

For a computational project, students should actually implement this and can also calculate quantities such as standard errors and confidence intervals.

22.12 Searching Over the Decision Variable

After estimating expected profit for $N = 101$, change N to 102 and repeat the entire simulation; the distribution of X changes because there are now 102 potential customers. Generate show-ups, calculate profits, and average to obtain the estimated expected profit associated with 102 reservations. Then try 103, 104, and so forth. Eventually the estimated expected-profit curve should reach a maximum: the process is one of repeatedly increasing N and looking for the point where expected profit stops increasing and starts declining. That turning point is the solution to the optimization-with-simulation problem.

22.13 Why Profit Eventually Declines

Initially, overbooking is profitable because the hotel has predictable no-shows: selling one additional reservation generates an additional \$150 of revenue, and most of the time there is still enough physical capacity because some customers do not appear. But as N increases, the probability that more than 100 customers arrive also increases, and once that happens frequently enough, the expected \$200 penalties begin to dominate the benefit of accepting additional reservations. There is a balance between the benefit of selling additional capacity and the expected cost of excessive arrivals, which simulation evaluates when the randomness makes direct reasoning cumbersome.

22.14 Simulation versus Optimization

This example makes an important conceptual distinction. The **simulation problem** asks: for a given N, what is the distribution of profit, or more specifically, what is the expected profit associated with this N? The **optimization problem** asks: which N gives the highest expected profit? Optimization with simulation is essentially using simulation

to evaluate the objective function, and then optimizing over the decision variable — a pattern that appears in many real-world problems where the objective function may be impossible or inconvenient to calculate analytically, but can be estimated numerically if the system can be simulated for any proposed decision.

22.15 What Is Required for the Final Exam

For a final-exam problem of this kind, two things are expected. First, **develop the simulator for the uncertain quantity** — in this example, explaining how to simulate the number of show-ups. Second, **show how the resulting state determines revenue, cost, or profit** — in this example, giving the piecewise profit expression. Those two pieces constitute the expected final-exam solution: you are not expected to run thousands of simulations, numerically search over all booking limits, or write code during the exam. The emphasis is on the correct simulation logic and the correct economic performance calculation.

22.16 What Is Required for the Group Assignment

The group assignment goes further: students must actually implement the simulator, using Python or another suitable computational environment, to generate many random show-up values, calculate profit for each observation, average the simulated profits, estimate expected profit, and potentially calculate a standard error and confidence interval. This is repeated for multiple reservation limits ($N = 101, 102, 103, \dots$), and the reservation limit producing the highest estimated expected profit is identified. This optimization and numerical implementation goes beyond what is required on the final exam.

22.17 A Compact Exam Template

When faced with a simulation-based business problem, it can help to memorize the structure rather than individual numbers, by asking four questions:

1. **What is the decision?** Here: the number of reservations N.
2. **What is random?** Here: the number of customers who show up, X.
3. **How do I simulate X?** Use transformation, acceptance–rejection, or Metropolis, as appropriate.
4. **Given X, how do I calculate the performance measure?** Here: profit as a function of N and X.

For the exam, those steps are usually the core answer. For a full computational project, a fifth question is added: **how do I compare decisions?** — run many simulations for every candidate N, estimate expected profit, and choose the largest.

22.18 The Broader Lesson

The hotel example is more than an overbooking problem; it illustrates a general framework for decision analytics under uncertainty. There is a **controllable decision**, an **uncertain outcome**, and an **economic consequence** produced jointly by the decision and the random outcome. Simulation handles the uncertainty; optimization handles the decision. Together they provide a way to solve complex problems in which expected performance cannot easily be calculated directly. The same structure appears in inventory problems, staffing problems, capacity planning, revenue management, transportation, financial risk, and many other settings.

Final Summary of Simulation Module

This module placed Metropolis alongside transformation and acceptance–rejection as the third major random-number-generation method. Metropolis constructs a Markov chain whose long-run visitation frequencies reproduce a desired target distribution, using the acceptance rule

$$\alpha = \min\left(1, \frac{\text{proposed-state score}}{\text{current-state score}}\right),$$

which is powerful because common normalization constants cancel. For discrete problems, state-transition probabilities are constructed explicitly and uniform random numbers determine actual transitions; for continuous problems, a random perturbation creates a proposed state and the same acceptance-ratio idea applies. Finally, the hotel overbooking problem demonstrates how simulation becomes part of optimization: for each booking limit, simulate show-ups, convert show-ups into profit, estimate expected profit, and compare alternative booking limits.

Simulation tells us what might happen under a decision; optimization with simulation tells us which decision we should choose.